

Python for AI/ML in CFD - Coding Test Preparation Handbook

Targeted at PINNs, MeshGraphNets and Scientific ML roles

Practice Questions, Reference Code and Mock Tests

- How to Use This Handbook
 - The four test formats you will actually face
 - The scoring model in the examiner's head
 - Suggested route through this document
- Part 1 — Core Python You Must Be Fluent In
 - 1.1 Data structures: choosing correctly under time pressure
 - 1.2 Comprehensions, generators and laziness
 - 1.3 Functions: arguments, closures, `functools`
 - 1.4 Decorators
 - 1.5 Context managers
 - 1.6 Object-oriented Python for solver/model architecture
 - 1.7 Type hints
 - 1.8 Errors, and floating point
 - 1.9 Practice set — Part 1
- Part 2 — NumPy: The Real Test
 - 2.1 The mental model
 - 2.2 Broadcasting
 - 2.3 Vectorisation patterns you must know cold
 - 2.4 `einsum`, `tensordot` and stencils
 - 2.5 Linear algebra
 - 2.6 Random numbers, correctly
 - 2.7 Performance rules
 - 2.8 Practice set — Part 2
- Part 3 — SciPy, Sparse Systems and Optimisation
 - 3.1 Sparse matrices — the language of implicit CFD
 - 3.2 ODE integration
 - 3.3 Optimisation and root finding
 - 3.4 Interpolation — mapping between meshes
 - 3.5 Signal processing on CFD time series
 - 3.6 Practice set — Part 3
- Part 4 — Data Handling for Simulation Datasets
 - 4.1 Pandas essentials
 - 4.2 File formats for simulation data
 - 4.3 `xarray` for labelled multi-dimensional fields
 - 4.4 Practice set — Part 4
- Part 5 — Algorithms and Data Structures That Still Appear
 - 5.1 The realistic scope
 - 5.2 Templates worth memorising
 - 5.3 Domain-flavoured DSA questions
- Part 6 — Numerical Methods and CFD in Pure Python
 - 6.1 Finite differences and the model equations
 - 6.2 Stability: the numbers you must recite
 - 6.3 Verification: the habit that impresses
 - 6.4 Practice set — Part 6
- Part 7 — Meshes, Geometry and Unstructured Data
 - 7.1 Reading meshes
 - 7.2 Mesh to graph — the MeshGraphNet preprocessing step

- 7.3 Geometric quantities
- 7.4 Signed distance and wall distance
- 7.5 Practice set — Part 7
- Part 8 — PyTorch Fundamentals
 - 8.1 Tensors and devices
 - 8.2 Autograd — the single most important topic for PINNs
 - 8.3 Models, losses, optimisers
 - 8.4 Datasets and DataLoaders
 - 8.5 Practice set — Part 8
- Part 9 — Physics-Informed Neural Networks (PINNs)
 - 9.1 The formulation
 - 9.2 Complete PINN: 1-D Burgers equation
 - 9.3 PINN for steady 2-D Navier-Stokes with a stream function
 - 9.4 Hard constraints — the trick that fixes most convergence problems
 - 9.5 Loss balancing — the number-one PINN failure mode
 - 9.6 Adaptive sampling
 - 9.7 The PINN failure catalogue — expect to be asked
 - 9.8 PINNs vs classical solvers — the judgement question
 - 9.9 Practice set — Part 9
- Part 10 — Graph Neural Networks and MeshGraphNets
 - 10.1 Why graphs for CFD
 - 10.2 Message passing from scratch (no PyG) — write this in an interview
 - 10.3 MeshGraphNet: encode-process-decode
 - 10.4 The same thing with PyTorch Geometric
 - 10.5 Related architectures you should be able to name
 - 10.6 Practice set — Part 10
- Part 11 — ML Engineering for Scientific Data
 - 11.1 Splitting data correctly — the most common take-home failure
 - 11.2 Normalisation
 - 11.3 Metrics for field prediction
 - 11.4 Reproducibility
 - 11.5 Overfitting, underfitting and what to do
 - 11.6 Practice set — Part 11
- Part 12 — Performance, Testing and Engineering Practice
 - 12.1 Profiling before optimising
 - 12.2 Speed-up options, in the order you should reach for them
 - 12.3 Testing numerical code
 - 12.4 Project layout and tooling
 - 12.5 Practice set — Part 12
- Part 13 — Conceptual Question Bank with Model Answers
 - 13.1 Python and software
 - 13.2 NumPy and numerics
 - 13.3 CFD and numerics
 - 13.4 Machine learning
 - 13.5 PINN and operator learning
 - 13.6 Graph networks
 - 13.7 Judgement and the questions with no code
- Part 14 — Timed Mock Tests
 - Mock Test A — Automated screen (60 minutes)
 - Mock Test B — Live pair-coding (45 minutes)
 - Mock Test C — Take-home (4 hours)
 - Mock Test D — Whiteboard (30 minutes)
- Part 15 — Cheat Sheets and Study Plan
 - 15.1 Numbers to have memorised
 - 15.2 Complexity table

- 15.3 One-page PyTorch recall list
- 15.4 Four-week preparation plan
- 15.5 Ten things to say that signal seniority
- 15.6 Self-assessment checklist

How to Use This Handbook

This handbook is built for one specific job: walking into a Python coding assessment for an **AI/ML-for-CFD** role (physics-informed neural networks, graph-network surrogates, data-driven turbulence modelling, ROM/surrogate modelling) and not being surprised by anything.

The four test formats you will actually face

Format	Duration	What they test	Where it appears
Automated screen (HackerRank / Codility / CoderPad)	45–90 min	Core Python, NumPy vectorisation, 1–2 easy/medium DSA problems, sometimes a small numerical task	Large OEMs, Tier-1 suppliers, big software vendors
Take-home task	3–8 h, or 3 days	Read simulation data, build a small model, evaluate it, write clean code + a README	Most common for scientific-ML roles
Live pair-coding	45–60 min	Reasoning out loud, incremental building, debugging, NumPy/PyTorch fluency	Research groups, startups, Siemens/Ansys/NVIDIA-style teams
Whiteboard / conceptual	30–45 min	Derivatives via autograd, loss design, message passing, discretisation, “why did the training diverge?”	Always, bundled into the technical round

The scoring model in the examiner’s head

Assessors for scientific-ML roles almost never score you purely on whether the code runs. A typical rubric:

Criterion	Weight	How to win it
Correctness	30%	Test edge cases yourself before submitting
Numerical/physical soundness	25%	Non-dimensionalise, check units, check conservation, check CFL
Vectorisation & performance	15%	No Python loops over cells/nodes; use NumPy/Torch ops
Code quality	15%	Type hints, docstrings, pure functions, no globals, one job per function
Communication	15%	Narrate assumptions; a short README beats extra features

The single biggest differentiator in this field: candidates who write `for i in range(n)` over mesh cells versus candidates who write array/tensor operations. Everything in Part 3 is worth more to you than another sorting algorithm.

Suggested route through this document

1. **Parts 1–3** (core Python, NumPy, SciPy) — the non-negotiable baseline. Do every exercise.
2. **Parts 4–5** (data handling, DSA) — one pass, then use as reference.
3. **Parts 6–7** (numerics, meshes) — this is where your CFD background becomes an unfair advantage.
4. **Parts 8–10** (PyTorch, PINNs, MeshGraphNet) — the core of the role. Type the code out, do not read it.
5. **Parts 11–12** (ML engineering, performance) — what separates a hire from a strong hire.
6. **Parts 13–15** (concept bank, mock tests, cheat sheets) — final two weeks.

Every code block in this handbook is written to be runnable with only `numpy`, `scipy`, `pandas`, `torch` and (where noted) `torch_geometric`, `meshio`, `pyvista`.

Part 1 — Core Python You Must Be Fluent In

1.1 Data structures: choosing correctly under time pressure

Structure	Lookup	Insert	Ordered	Use it when
list	O(n)	O(1) append	yes	Sequence, index access, stack
dict	O(1)	O(1)	insertion order	Mapping, memoisation, counting, grouping
set	O(1)	O(1)	no	Membership tests, dedup, node/face uniqueness
tuple	O(n)	immutable	yes	Hashable keys (e.g. a sorted face (n1,n2,n3))
collections.deque	O(n)	O(1) both ends	yes	BFS queue, sliding window, rolling buffer
collections.defaultdict	O(1)	O(1)	insertion	Adjacency lists, grouping without key checks
collections.Counter	O(1)	O(1)	insertion	Frequency, most-common
heapq (list)	—	O(log n)	no	Priority queue, top-k, Dijkstra

Interview-standard idiom: building mesh connectivity

```
from collections import defaultdict

def build_node_to_cells(cells: list[tuple[int, ...]]) -> dict[int, list[int]]:
    """Map each node id to the list of cell ids that contain it."""
    node_to_cells = defaultdict(list)
    for cid, cell in enumerate(cells):
        for node in cell:
            node_to_cells[node].append(cid)
    return dict(node_to_cells)

def find_boundary_faces(tets: list[tuple[int, int, int, int]]) -> list[tuple[int, ...]]:
    """A face is on the boundary if exactly one cell owns it."""
    from collections import Counter
    face_count = Counter()
    for t in tets:
        for face in ((t[0], t[1], t[2]), (t[0], t[1], t[3]),
                     (t[0], t[2], t[3]), (t[1], t[2], t[3])):
            face_count[tuple(sorted(face))] += 1
    return [f for f, c in face_count.items() if c == 1]
```

This exact question — “given a tet mesh connectivity list, extract the surface” — appears in real CFD-adjacent interviews. The trick is **sorting the face tuple** so orientation does not matter.

1.2 Comprehensions, generators and laziness

```

# List comprehension with condition
squares = [x**2 for x in range(20) if x % 3 == 0]

# Dict comprehension - invert a mapping
field_units = {"U": "m/s", "p": "Pa", "T": "K"}
unit_to_field = {v: k for k, v in field_units.items()}

# Set comprehension
unique_bcs = {patch.split("_")[0] for patch in ["inlet_1", "inlet_2", "wall_a"]}

# Nested comprehension (row-major flattening)
matrix = [[1, 2], [3, 4], [5, 6]]
flat = [x for row in matrix for x in row]          # [1,2,3,4,5,6]

# Generator expression - lazy, O(1) memory. Critical for large time-series files.
total_ke = sum(0.5 * rho * u**2 for rho, u in zip(rho_list, u_list))

```

Generators for streaming large simulation output

```

from pathlib import Path
from typing import Iterator
import numpy as np

def stream_timesteps(folder: Path) -> Iterator[tuple[float, np.ndarray]]:
    """Yield (time, field) one snapshot at a time - never loads all data into RAM."""
    for f in sorted(folder.glob("t_*.npz"), key=lambda p: float(p.stem.split("_")[1])):
        t = float(f.stem.split("_")[1])
        yield t, np.load(f)

# Consume lazily
for t, field in stream_timesteps(Path("run01")):
    if np.isnan(field).any():
        raise ValueError(f"NaN detected at t={t}")

```

Likely question: “What is the difference between a list comprehension and a generator expression?”

Answer: the comprehension materialises all elements immediately (O(n) memory, re-iterable); the generator produces on demand (O(1) memory, single-pass, lazily evaluated). For 500 GB of transient CFD output, only the generator is viable.

1.3 Functions: arguments, closures, `functools`

```

def solve(mesh, *, tol: float = 1e-6, max_iter: int = 1000, verbose: bool = False):
    """Keyword-only args after * - prevents accidental positional mistakes."""
    ...

# Mutable default argument - THE classic Python trap
def bad(history=[]):          # WRONG: the list is created once, shared by every call
    history.append(1)
    return history

def good(history=None):       # CORRECT
    history = [] if history is None else history
    history.append(1)
    return history

```

```

from functools import lru_cache, partial, reduce
import operator

@lru_cache(maxsize=None)
def shape_function_weights(order: int) -> tuple[float, ...]:
    """Memoised - expensive quadrature weights computed once per order."""
    ...

# partial: freeze arguments to build a residual function for a solver
residual_at_re = partial(navier_stokes_residual, reynolds=1000.0)

# reduce: fold - rarely needed but occasionally asked
product = reduce(operator.mul, [1, 2, 3, 4], 1)  # 24

```

1.4 Decorators

Decorators come up constantly, both as “explain this” and “write one”.

```

import time
import functools

def timeit(func):
    """Log wall-clock time of a function. functools.wraps preserves metadata."""
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__}: {time.perf_counter() - t0:.4f} s")
        return result
    return wrapper

def retry(n_attempts: int = 3, delay: float = 1.0):
    """Decorator WITH arguments - needs three levels of nesting."""
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(n_attempts):
                try:
                    return func(*args, **kwargs)
                except Exception as exc:
                    if attempt == n_attempts - 1:
                        raise
                    print(f"attempt {attempt + 1} failed: {exc}")
                    time.sleep(delay)
            return wrapper
        return decorator

@timeit
@retry(n_attempts=2)
def run_solver(case: str) -> float:
    ...

```

Exam question: “Why do you need `functools.wraps`?” Without it the wrapper replaces `__name__`, `__doc__` and the signature of the original function, which breaks introspection, documentation tooling and pickling — the last of which matters for `multiprocessing`.

1.5 Context managers

```

from contextlib import contextmanager
import numpy as np

@contextmanager
def numpy_error_state(**kwargs):
    """Temporarily change how NumPy reports floating point problems."""
    old = np.seterr(**kwargs)
    try:
        yield
    finally:
        np.seterr(**old) # restored even if the body raises

with numpy_error_state(divide="raise", invalid="raise"):
    ratio = flux / area # now raises instead of producing silent inf/nan

```

Class-based equivalent (they may ask for this form specifically):

```

class Timer:
    def __enter__(self):
        self.t0 = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        self.elapsed = time.perf_counter() - self.t0
        return False # False -> do not suppress exceptions

```

1.6 Object-oriented Python for solver/model architecture

```

from abc import ABC, abstractmethod
from dataclasses import dataclass, field
import numpy as np

@dataclass
class FlowState:
    """dataclass gives __init__, __repr__, __eq__ for free."""
    u: np.ndarray
    p: np.ndarray
    t: float = 0.0
    metadata: dict = field(default_factory=dict) # never use mutable default directly

    @property
    def kinetic_energy(self) -> float:
        return 0.5 * float(np.sum(self.u ** 2))

class Discretisation(ABC):
    """Abstract base - forces subclasses to implement the interface."""
    @abstractmethod
    def gradient(self, phi: np.ndarray) -> np.ndarray: ...

    @abstractmethod
    def laplacian(self, phi: np.ndarray) -> np.ndarray: ...

    def diffuse(self, phi: np.ndarray, nu: float, dt: float) -> np.ndarray:
        """Template method - shared logic built on the abstract parts."""
        return phi + nu * dt * self.laplacian(phi)

class CentralDifference2D(Discretisation):
    def __init__(self, dx: float, dy: float):
        self.dx, self.dy = dx, dy

    def gradient(self, phi):
        return np.gradient(phi, self.dy, self.dx)

    def laplacian(self, phi):
        d2x = (np.roll(phi, -1, 1) - 2 * phi + np.roll(phi, 1, 1)) / self.dx ** 2
        d2y = (np.roll(phi, -1, 0) - 2 * phi + np.roll(phi, 1, 0)) / self.dy ** 2
        return d2x + d2y

```

Key OOP facts they probe:

Concept	One-line answer
<code>@classmethod</code> vs <code>@staticmethod</code>	<code>classmethod</code> receives <code>cls</code> (alternative constructors); <code>staticmethod</code> receives nothing (namespaced utility)
<code>__slots__</code>	Removes per-instance <code>__dict__</code> ; big memory saving for millions of small objects
MRO	Method resolution order, C3 linearisation; inspect via <code>Class.__mro__</code>
<code>super()</code>	Delegates to next class in the MRO, not necessarily the “parent”
Duck typing	If it implements the protocol, it works — no inheritance required
<code>@property</code>	Computed attribute with validation, no API change for callers

1.7 Type hints

Scientific-ML teams increasingly gate merges on `mypy`. Show that you know this.


```

from typing import Callable, Sequence, TypeAlias
import numpy as np
import numpy.typing as npt

Array: TypeAlias = npt.NDArray[np.float64]

def rk4_step(
    f: Callable[[float, Array], Array],
    t: float,
    y: Array,
    dt: float,
) -> Array:
    """Classical fourth-order Runge-Kutta step."""
    k1 = f(t, y)
    k2 = f(t + 0.5 * dt, y + 0.5 * dt * k1)
    k3 = f(t + 0.5 * dt, y + 0.5 * dt * k2)
    k4 = f(t + dt, y + dt * k3)
    return y + dt / 6.0 * (k1 + 2 * k2 + 2 * k3 + k4)

```

1.8 Errors, and floating point

```

class ConvergenceError(RuntimeError):
    """Domain-specific exception - always better than bare RuntimeError."""
    def __init__(self, residual: float, iteration: int):
        super().__init__(f"diverged: residual={residual:.3e} at iter {iteration}")
        self.residual, self.iteration = residual, iteration

try:
    solve()
except ConvergenceError as exc:
    logger.warning("retrying with smaller dt: %s", exc)
except (ValueError, KeyError) as exc:
    raise RuntimeError("bad case setup") from exc    # preserve the cause
else:
    logger.info("converged")
finally:
    cleanup()

```

Floating point questions are common in numerical roles:

```

0.1 + 0.2 == 0.3          # False
abs((0.1 + 0.2) - 0.3) < 1e-12  # True - always compare with a tolerance
np.isclose(0.1 + 0.2, 0.3)      # rtol=1e-5, atol=1e-8 by default
np.allclose(a, b, rtol=1e-7)    # arrays

import math
math.isnan(float("nan"))        # True; note nan != nan
np.finfo(np.float64).eps       # 2.22e-16 machine epsilon
np.finfo(np.float32).eps       # 1.19e-07 - relevant for mixed-precision training

```

Question you will get: “Your residual stalls at 1e-8 in single precision. Why?” Because float32 has ~7 decimal digits; you have reached the noise floor of the arithmetic. Either move to float64, or normalise the residual by a reference value.

1.9 Practice set — Part 1

Q1.1 Write a function that, given a list of `(patch_name, face_ids)` tuples, returns a dict mapping each face id to its patch. Raise if a face appears in two patches.

Q1.2 Implement an LRU cache class from scratch (no `functools`) with $O(1)$ `get` and `put`. *Hint: dict + doubly linked list, or `collections.OrderedDict` with `move_to_end`.*

Q1.3 Write a decorator `@validate_shape(expected_ndim=2)` that raises `ValueError` if the first positional argument is not a NumPy array of the given dimensionality.

Q1.4 Given a generator of residual values, write a function that returns the first index where the residual drops below `tol` and stays below it for `k` consecutive iterations — without materialising the sequence.

Q1.5 Explain what this prints and why:

```

funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])

```

Answer: [2, 2, 2] — late binding; `i` is looked up when the lambda is called. Fix with `lambda i=i: i`.

Selected solutions

```
# Q1.1
def face_to_patch(patches: list[tuple[str, list[int]]]) -> dict[int, str]:
    mapping: dict[int, str] = {}
    for name, faces in patches:
        for f in faces:
            if f in mapping:
                raise ValueError(f"face {f} in both {mapping[f]!r} and {name!r}")
            mapping[f] = name
    return mapping

# Q1.2
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self._d: OrderedDict = OrderedDict()

    def get(self, key):
        if key not in self._d:
            return None
        self._d.move_to_end(key)
        return self._d[key]

    def put(self, key, value):
        if key in self._d:
            self._d.move_to_end(key)
        self._d[key] = value
        if len(self._d) > self.capacity:
            self._d.popitem(last=False)

# Q1.3
import functools, numpy as np

def validate_shape(expected_ndim: int):
    def deco(func):
        @functools.wraps(func)
        def wrapper(arr, *args, **kwargs):
            if not isinstance(arr, np.ndarray) or arr.ndim != expected_ndim:
                raise ValueError(
                    f"{func.__name__} expects a {expected_ndim}D array, "
                    f"got {type(arr).__name__} with ndim="
                    f"{getattr(arr, 'ndim', None)}"
                )
            return func(arr, *args, **kwargs)
        return wrapper
    return deco

# Q1.4
def first_stable_index(residuals, tol: float, k: int) -> int | None:
    run_start, run_len = None, 0
    for i, r in enumerate(residuals):
        if r < tol:
            if run_len == 0:
                run_start = i
            run_len += 1
            if run_len == k:
                return run_start
    else:
        run_len = 0
    return None
```

Part 2 — NumPy: The Real Test

For an AI/ML-in-CFD role, NumPy fluency is weighted higher than everything else in Part 1. Expect at least one task where the naive loop solution is rejected.

2.1 The mental model

An `ndarray` is a **contiguous memory buffer** plus metadata: `shape`, `dtype`, `strides`. Slicing returns a *view* (no copy, shares memory); fancy indexing returns a *copy*.

```
import numpy as np

a = np.arange(24, dtype=np.float64).reshape(2, 3, 4)
a.shape      # (2, 3, 4)
a.strides    # (96, 32, 8) bytes to step in each dimension
a.nbytes     # 192
a.flags['C_CONTIGUOUS'] # True (row-major)

b = a[:, 1, :]      # VIEW -> writing to b modifies a
c = a[[0, 1], 1, :] # COPY (fancy indexing)
b.base is a        # True -> quick way to check if something is a view
```

Operation	View or copy?
Basic slice <code>a[1:5, ::2]</code>	View
<code>a.reshape(...)</code>	View if compatible strides, else copy
<code>a.T</code> , <code>np.transpose</code>	View
<code>a.ravel()</code>	View if possible
<code>a.flatten()</code>	Always copy
Boolean mask <code>a[a > 0]</code>	Copy
Integer/fancy index <code>a[[0,2]]</code>	Copy
<code>np.ascontiguousarray</code>	Copy if not already contiguous

Why it matters: an accidental copy of a 2 GB field array inside a time loop is the most common performance bug in scientific Python.

2.2 Broadcasting

Rules, applied right-to-left: dimensions are compatible if equal, or one of them is 1, or one is missing.

```
u = np.random.rand(100_000, 3) # velocity at 100k cells
scale = np.array([1.0, 1.0, 0.5]) # (3,) -> broadcast over rows
u_scaled = u * scale # (100000, 3)

cell_volume = np.random.rand(100_000) # (100000,)
# u * cell_volume -> ERROR: (100000,3) vs (100000,)
weighted = u * cell_volume[:, None] # (100000,1) -> OK

# Pairwise distances without any loop (careful: 0(n^2) memory)
pts = np.random.rand(500, 3)
d = np.linalg.norm(pts[:, None, :] - pts[None, :, :], axis=-1) # (500, 500)
```

Classic live-coding request: pairwise distances, memory-efficient version.

```
def pairwise_sq_dist(a: np.ndarray, b: np.ndarray) -> np.ndarray:
    """||a-b||^2 = |a|^2 - 2 a.b + |b|^2 -- 0(n*m) memory instead of 0(n*m*d)."""
    a2 = np.einsum("ij,ij->i", a, a)[:, None]
    b2 = np.einsum("ij,ij->i", b, b)[None, :]
    return np.maximum(a2 - 2.0 * a @ b.T + b2, 0.0) # clip negative round-off
```

2.3 Vectorisation patterns you must know cold

```

# 1. Replace if/else on arrays
tau = np.where(y_plus < 5.0, mu * u / y, rho * u_tau**2)

# 2. Multi-branch: piecewise
nu_t = np.piecewise(y_plus,
                    [y_plus < 5, (y_plus >= 5) & (y_plus < 30), y_plus >= 30],
                    [lambda x: 0.0 * x, lambda x: 0.01 * x, lambda x: 0.41 * x])

# 3. Masked reduction
mean_inside = field[mask].mean()
mean_inside = np.where(mask, field, np.nan)
mean_inside = np.nanmean(mean_inside)

# 4. Cumulative operations
cum_flux = np.cumsum(flux * dA)

# 5. Sorting / ranking
idx = np.argsort(residual)[: -1][:10] # 10 worst cells
top_k = np.argpartition(residual, -10)[-10:] # O(n), faster when order not needed

# 6. Unique and grouping
zone_ids, inverse, counts = np.unique(zones, return_inverse=True, return_counts=True)

# 7. Segment sum - accumulate cell values onto nodes (the CFD workhorse)
node_sum = np.zeros(n_nodes)
np.add.at(node_sum, node_index, cell_value) # correct with duplicates, slow
node_sum = np.bincount(node_index, weights=cell_value, minlength=n_nodes) # fast

# 8. searchsorted - bin/locate values in a monotonic array
bin_idx = np.searchsorted(x_faces, x_probe, side="right") - 1

```

np.add.at VS np.bincount: both handle repeated indices correctly (plain `arr[idx] += val` does **not** — it silently drops duplicates). `bincount` is far faster for 1-D scatter-add. For multi-component fields, loop over the 3 components or use `np.add.at`. This exact question — “why does `arr[idx] += 1` give the wrong answer?” — is a favourite.

2.4 einsum, tensordot and stencils

```

# Contract a stress tensor field with normals: t_i = sigma_ij n_j
sigma = np.random.rand(N, 3, 3)
n     = np.random.rand(N, 3)
trac  = np.einsum("nij,nj->ni", sigma, n)

# Frobenius norm of strain rate at every cell: S = sqrt(2 S_ij S_ij)
S = 0.5 * (grad_u + np.swapaxes(grad_u, -1, -2))
S_mag = np.sqrt(2.0 * np.einsum("nij,nij->n", S, S))

# Trace, deviatoric part
tr = np.einsum("nii->n", sigma)
dev = sigma - (tr / 3.0)[ :, None, None] * np.eye(3)

```

Sliding windows without loops

```

from numpy.lib.stride_tricks import sliding_window_view

phi = np.random.rand(256, 256)
w = sliding_window_view(phi, (3, 3)) # shape (254, 254, 3, 3), zero copy
laplace = (w * np.array([[0, 1, 0], [1, -4, 1], [0, 1, 0]]).sum((-1, -2))

```

2.5 Linear algebra

```

A = np.random.rand(200, 200)
b = np.random.rand(200)

x = np.linalg.solve(A, b) # LU. NEVER use inv(A) @ b
x, *_ = np.linalg.lstsq(A, b, rcond=None) # least squares / overdetermined
Q, R = np.linalg.qr(A)
U, s, Vt = np.linalg.svd(A, full_matrices=False) # POD / PCA / ROM backbone
w, V = np.linalg.eigh(A + A.T) # eigh for symmetric - faster and real
cond = np.linalg.cond(A) # ill-conditioning check

```

Proper Orthogonal Decomposition (POD) — asked constantly in CFD-ML interviews:

```
def pod(snapshots: np.ndarray, n_modes: int):
    """snapshots: (n_dof, n_snapshots). Returns modes, temporal coeffs, energy fraction."""
    mean = snapshots.mean(axis=1, keepdims=True)
    X = snapshots - mean
    U, s, Vt = np.linalg.svd(X, full_matrices=False)
    energy = np.cumsum(s ** 2) / np.sum(s ** 2)
    return U[:, :n_modes], (np.diag(s) @ Vt)[:n_modes], energy[:n_modes]
```

Follow-up they will ask: “What if `n_dof` is 10 million and you have 500 snapshots?” Use the **method of snapshots**: form the 500x500 correlation matrix `X.T @ X`, eigendecompose that, and recover spatial modes as `X @ V / s`. Cost drops from $O(n \cdot m^2)$ on a huge matrix to something trivial.

2.6 Random numbers, correctly

```
rng = np.random.default_rng(seed=42) # modern API - use this, not np.random.seed
noise = rng.normal(0.0, 0.01, size=field.shape)
idx = rng.choice(n, size=1000, replace=False)
rng.shuffle(indices)
```

2.7 Performance rules

Rule	Why
Preallocate with <code>np.empty</code> / <code>np.zeros</code> , never <code>np.append</code> in a loop	<code>append</code> reallocates and copies every call — $O(n^2)$
Use in-place ops (<code>a += b</code> , <code>np.add(a, b, out=a)</code>)	Avoids temporary arrays
Keep dtype consistent; avoid silent upcast to float64	Halves memory in float32 pipelines
Prefer contiguous access along the last axis	Cache locality; check <code>.flags</code>
Use <code>np.einsum(..., optimize=True)</code> for 3+ operands	Chooses a good contraction order
Chunk large reductions	Avoids the $O(n^2)$ intermediate in pairwise ops

```
# Bad
result = []
for i in range(n):
    result.append(f(x[i]))
result = np.array(result)

# Good
result = f(x) # if f is vectorised
# or
result = np.empty(n)
np.sin(x, out=result) # zero temporaries
```

2.8 Practice set — Part 2

Q2.1 Given `coords` of shape `(N, 3)` and a bounding box, return the indices of points inside it — one line, no loops.

Q2.2 Given cell-centred scalar `phi` of shape `(nz, ny, nx)` on a uniform grid, compute the magnitude of the gradient with second-order central differences in the interior and one-sided differences at the boundaries. Do not use `np.gradient` — implement it.

Q2.3 You have `face_owner` (int array, length `n_faces`) and `face_flux` (float, length `n_faces`). Compute the net flux out of every cell. Handle repeated owners correctly.

Q2.4 Given a `(N, 3, 3)` array of velocity-gradient tensors, compute the Q-criterion $Q = 0.5 * (|\Omega|^2 - |S|^2)$ for every cell, fully vectorised.

Q2.5 Downsample a `(1024, 1024)` field to `(256, 256)` by 4x4 block averaging without any Python loop.

Q2.6 Explain why `a[idx] += 1` under-counts when `idx` contains duplicates, and give two fixes.

Solutions

```

import numpy as np

# Q2.1
inside = np.flatnonzero(np.all((coords >= lo) & (coords <= hi), axis=1))

# Q2.2
def grad_mag(phi: np.ndarray, dz: float, dy: float, dx: float) -> np.ndarray:
    def d_axis(f, h, ax):
        g = np.empty_like(f)
        sl = [slice(None)] * f.ndim
        c, p, m = sl.copy(), sl.copy(), sl.copy()
        c[ax], p[ax], m[ax] = slice(1, -1), slice(2, None), slice(0, -2)
        g[tuple(c)] = (f[tuple(p)] - f[tuple(m)]) / (2 * h)
        f0, f1 = sl.copy(), sl.copy()
        f0[ax], f1[ax] = 0, 1
        g[tuple(f0)] = (f[tuple(f1)] - f[tuple(f0)]) / h          # forward
        l0, l1 = sl.copy(), sl.copy()
        l0[ax], l1[ax] = -1, -2
        g[tuple(l0)] = (f[tuple(l0)] - f[tuple(l1)]) / h          # backward
        return g
    gz, gy, gx = d_axis(phi, dz, 0), d_axis(phi, dy, 1), d_axis(phi, dx, 2)
    return np.sqrt(gz**2 + gy**2 + gx**2)

# Q2.3
net_flux = np.bincount(face_owner, weights=face_flux, minlength=n_cells)

# Q2.4
def q_criterion(grad_u: np.ndarray) -> np.ndarray:
    S = 0.5 * (grad_u + np.swapaxes(grad_u, -1, -2))
    W = 0.5 * (grad_u - np.swapaxes(grad_u, -1, -2))
    s2 = np.einsum("nij,nij->n", S, S)
    w2 = np.einsum("nij,nij->n", W, W)
    return 0.5 * (w2 - s2)

# Q2.5
coarse = field.reshape(256, 4, 256, 4).mean(axis=(1, 3))

# Q2.6
# a[idx] += 1 expands to a[idx] = a[idx] + 1: the RHS is evaluated once against the
# ORIGINAL a, so duplicate indices all write the same value - last write wins.
np.add.at(a, idx, 1)          # fix 1: unbuffered, correct, slower
a += np.bincount(idx, minlength=a.size)  # fix 2: fast for 1-D integer scatter

```

Part 3 — SciPy, Sparse Systems and Optimisation

3.1 Sparse matrices — the language of implicit CFD

Format	Best for
<code>coo_matrix</code>	Assembly from (row, col, val) triplets; duplicates summed on conversion
<code>csr_matrix</code>	Row slicing, matrix-vector products, iterative solvers
<code>csc_matrix</code>	Column slicing, direct factorisation
<code>dia_matrix</code>	Structured/banded grids
<code>lil_matrix</code>	Incremental construction with changing sparsity (slow, avoid in loops)

```
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla

def laplacian_1d(n: int, h: float) -> sp.csr_matrix:
    """Second-order Dirichlet Laplacian: tridiagonal (1, -2, 1)/h^2."""
    main = -2.0 * np.ones(n)
    off = np.ones(n - 1)
    A = sp.diags([off, main, off], [-1, 0, 1], format="csr") / h**2
    return A

def laplacian_2d(nx: int, ny: int, h: float) -> sp.csr_matrix:
    """Kronecker sum - the standard trick, worth memorising."""
    Lx, Ly = laplacian_1d(nx, h), laplacian_1d(ny, h)
    Ix, Iy = sp.identity(nx), sp.identity(ny)
    return sp.kron(Iy, Lx) + sp.kron(Ly, Ix)

A = laplacian_2d(128, 128, 1.0 / 128)
b = np.random.rand(A.shape[0])

x_direct = spla.spsolve(A.tocsc(), b)           # sparse LU
x_cg, info = spla.cg(A, b, rtol=1e-8, maxiter=2000) # symmetric positive definite
x_gm, info = spla.gmres(A, b, restart=50, rtol=1e-8) # non-symmetric

# Preconditioning - the question that separates real users from tutorial readers
ilu = spla.spilu(A.tocsc(), drop_tol=1e-5, fill_factor=10)
M = spla.LinearOperator(A.shape, ilu.solve)
x_pcg, info = spla.pcg(A, b, M=M, rtol=1e-8)
```

Assembly pattern for an unstructured mesh (they love this one):

```
rows, cols, vals = [], [], []
for owner, neigh, coeff in face_data:
    rows += [owner, owner, neigh, neigh]
    cols += [owner, neigh, neigh, owner]
    vals += [coeff, -coeff, coeff, -coeff]
A = sp.coo_matrix((vals, (rows, cols)), shape=(n_cells, n_cells)).tocsr()
```

COO sums duplicates automatically on conversion — that is exactly what FV assembly needs.

3.2 ODE integration

```
from scipy.integrate import solve_ivp

def lorenz(t, y, sigma=10.0, rho=28.0, beta=8/3):
    x, yy, z = y
    return [sigma * (yy - x), x * (rho - z) - yy, x * yy - beta * z]

sol = solve_ivp(lorenz, t_span=(0, 40), y0=[1.0, 1.0, 1.0],
                method="RK45", rtol=1e-8, atol=1e-10,
                t_eval=np.linspace(0, 40, 4000), dense_output=True)
```

Method	Use when
RK45	Default, non-stiff
DOP853	High accuracy, non-stiff
Radau , BDF	Stiff systems — chemistry, fast/slow scales, fine grids
LSODA	Automatic stiff/non-stiff switching

Stiffness question: “How do you know a system is stiff?” The explicit solver takes vanishingly small steps despite a smooth solution; the Jacobian has eigenvalues spread over many orders of magnitude. Fix: implicit method (BDF / Radau) and supply the analytic Jacobian via `jac=`.

3.3 Optimisation and root finding

```
from scipy.optimize import minimize, least_squares, curve_fit, brentq, root

# Calibrate turbulence-model constants against reference data
def residuals(params, x_data, y_data):
    return model(x_data, *params) - y_data

res = least_squares(residuals, x0=[0.09, 1.44, 1.92], args=(x, y),
                    bounds=([0, 0, 0], [1, 5, 5]), loss="soft_l1")

# Constrained minimisation
res = minimize(objective, x0, method="SLSQP", jac=grad,
               constraints=[{"type": "ineq", "fun": lambda p: p[0] - 0.01}],
               bounds=[(0, 1)] * len(x0), options={"maxiter": 500})

# Scalar root: e.g. friction factor from Colebrook
f = brentq(colebrook, 1e-4, 1.0, args=(Re, rel_rough), xtol=1e-12)
```

3.4 Interpolation — mapping between meshes

```
from scipy.interpolate import (RegularGridInterpolator, griddata,
                              RBFInterpolator, interpId)
from scipy.spatial import cKDTree, Delaunay

# Structured source grid -> arbitrary probe points
interp = RegularGridInterpolator((z, y, x), field, method="linear",
                                bounds_error=False, fill_value=None)
values = interp(probe_points) # (M, 3)

# Unstructured -> unstructured (slow for large N, but the go-to for one-off mapping)
mapped = griddata(src_pts, src_val, dst_pts, method="linear", fill_value=0.0)

# Nearest-neighbour mapping, fast, the workhorse for CFD-to-ML datasets
tree = cKDTree(src_pts)
dist, idx = tree.query(dst_pts, k=1)
mapped = src_val[idx]

# Inverse-distance weighting with k neighbours
dist, idx = tree.query(dst_pts, k=8)
w = 1.0 / np.maximum(dist, 1e-12)
mapped = (w * src_val[idx]).sum(1) / w.sum(1)

# Radial basis functions - smooth, good for scattered sensor data
rbf = RBFInterpolator(src_pts, src_val, neighbors=64, kernel="thin_plate_spline")
```

3.5 Signal processing on CFD time series


```

from scipy import signal, fft

# Power spectral density of a probe signal - vortex shedding frequency / Strouhal number
f, Pxx = signal.welch(u_probe, fs=1.0 / dt, nperseg=4096, window="hann")
f_peak = f[np.argmax(Pxx)]
St = f_peak * D / U_inf

# Low-pass filter (zero phase shift - important, do not use lfilter)
sos = signal.butter(4, 500.0, "low", fs=1.0 / dt, output="sos")
u_filt = signal.sosfiltfilt(sos, u_probe)

# 2-D FFT for energy spectrum of isotropic turbulence
uhat = fft.fftn(u, axes=(0, 1, 2))
E_k = 0.5 * np.abs(uhat) ** 2

```

3.6 Practice set — Part 3

Q3.1 Assemble and solve the 2-D Poisson equation $-\nabla^2 p = f$ on a unit square with Dirichlet BCs using sparse matrices, and verify second-order convergence by refining the mesh.

Q3.2 Given scattered probe data (x, y, u) , build an interpolator that returns u at arbitrary points and gracefully extrapolates with the nearest value outside the hull.

Q3.3 Integrate a stiff 2-species reaction system; show that RK45 fails and BDF succeeds, and report the step counts.

Q3.4 Given a lift-coefficient time history, find the dominant shedding frequency and estimate the Strouhal number. Handle a non-uniform time step.

Q3.5 Implement Jacobi, Gauss-Seidel and SOR for the 1-D Poisson problem and compare iteration counts. Explain the optimal SOR relaxation factor.

Part 4 — Data Handling for Simulation Datasets

Take-home tasks almost always start with “here is some data, load it”. Losing 40 minutes to I/O is a real risk.

4.1 Pandas essentials

```
import pandas as pd

df = pd.read_csv("forces.dat", sep=r"\s+", comment="#",
                 names=["t", "cd", "cl", "cm"], engine="python")

df = df.assign(cd_smooth=lambda d: d.cd.rolling(200, center=True, min_periods=1).mean())

# Filter -> group -> aggregate (the standard interview chain)
summary = (df.query("t > 5.0")
           .groupby(pd.cut(df.t, bins=10))
           .agg(cd_mean=("cd", "mean"),
               cd_std=("cd", "std"),
               cl_rms=("cl", lambda s: float(np.sqrt((s**2).mean()))))
           .reset_index())

# Merge design-of-experiments table with results
doe = pd.read_csv("doe.csv")
merged = doe.merge(results, on="case_id", how="left", validate="one_to_one")

# Wide <-> long
long = df.melt(id_vars="t", value_vars=["cd", "cl"], var_name="coef", value_name="value")
wide = long.pivot(index="t", columns="coef", values="value")
```

Task	Idiom
Missing data	<code>df.isna().sum()</code> , <code>df.dropna(subset=[...])</code> , <code>df.fillna(method="ffill")</code>
Type memory	<code>df.astype({"case_id": "int32", "zone": "category"})</code>
Vectorised conditional	<code>np.select([c1, c2], [v1, v2], default=v3)</code>
Row-wise op	Avoid <code>df.apply(axis=1)</code> ; vectorise or use <code>np.vectorize</code> as last resort
Time indexing	<code>df.set_index(pd.to_datetime(df.t, unit="s")).resample("1s").mean()</code>
Chunked read	<code>pd.read_csv(f, chunksize=1_000_000)</code> then concatenate reductions

Common trap: `SettingWithCopyWarning`. It means you sliced a `DataFrame` and then wrote to the slice, which may or may not propagate. Fix by using `.loc[mask, col] = value` on the original, or by taking an explicit `.copy()`.

4.2 File formats for simulation data

Format	Read with	Notes
CSV / DAT	pandas, np.loadtxt	Human readable, slow, lossy for floats — always specify precision
NPY / NPZ	np.load	Fast, exact, NumPy-only
HDF5	h5py, pandas.read_hdf	Hierarchical, chunked, compressed, partial reads — the standard for large ML datasets
VTK / VTU / VTM	meshio, pyvista	Mesh + fields; what most CFD codes export
CGNS	h5py (it is HDF5), pyCGNS	Standard CFD data model
MF4 / MDF	asammdf, ETAS MDA	Measurement data, automotive
Parquet	pandas, pyarrow	Columnar, compressed, excellent for tabular DOE results
Zarr	zarr, xarray	Chunked, cloud/parallel friendly, good for huge 4-D fields

```
import h5py, numpy as np

# Writing a training dataset the right way: chunked + compressed
with h5py.File("dataset.h5", "w") as f:
    f.attrs["reynolds"] = 1000.0
    f.attrs["created"] = "2026-07-31"
    g = f.create_group("case_042")
    g.create_dataset("coords", data=coords.astype(np.float32))
    g.create_dataset("U", data=U.astype(np.float32),
                     chunks=(1024, 3), compression="gzip", compression_opts=4)

# Reading only what you need - no full load
with h5py.File("dataset.h5", "r") as f:
    subset = f["case_042/U"][10_000:20_000, :] # partial read straight off disk
```

4.3 xarray for labelled multi-dimensional fields

```
import xarray as xr

ds = xr.Dataset(
    data_vars={"u": (("t", "z", "y", "x"), u_data),
               "p": (("t", "z", "y", "x"), p_data)},
    coords={"t": times, "z": z, "y": y, "x": x},
    attrs={"Re": 3900, "solver": "pimpleFoam"},
)

ds.sel(t=slice(5, 10)).mean("t") # time-average a window
ds.u.isel(z=32).plot() # mid-plane slice
ds.to_netcdf("field.nc")
ds_mean = ds.coarsen(x=4, y=4, boundary="trim").mean() # downsample
```

xarray is worth mentioning in an interview: it makes the “which axis was time again?” class of bug impossible, which matters when you are feeding 4-D fields into a network.

4.4 Practice set — Part 4

Q4.1 Read a whitespace-delimited force file with a variable-length comment header, compute the mean and RMS of `cl` over the last 30% of the signal, and report the 95% confidence interval of the mean accounting for autocorrelation.

Q4.2 Given 200 case directories each containing `postProcessing/forces/0/coefficient.dat`, build a single tidy DataFrame with a `case_id` column, using a generator so memory stays flat.

Q4.3 Convert a folder of `.vtu` time steps into a single compressed HDF5 file with shape `(n_times, n_nodes, n_fields)` plus coordinate and connectivity datasets.

Part 5 — Algorithms and Data Structures That Still Appear

Screening platforms do not care that you do CFD. Budget one week for this section; you need speed, not depth.

5.1 The realistic scope

Topic	Likelihood	Canonical problems
Hash maps / counting	Very high	Two-sum, group anagrams, first unique element
Two pointers / sliding window	High	Longest substring, container with most water, max window sum
Sorting + custom keys	High	Merge intervals, meeting rooms, sort by multiple criteria
Binary search	High	Search rotated array, first bad version, search on the answer
BFS / DFS on graphs & grids	High (relevant to meshes!)	Number of islands, flood fill, connected components, topological sort
Heaps	Medium	Top-k frequent, k-way merge, median of a stream
Dynamic programming	Medium	Climbing stairs, coin change, LCS, edit distance
Trees	Medium	Traversals, level order, LCA
Union-Find	Medium (mesh partitioning!)	Connected components, cycle detection
Backtracking	Low	Permutations, subsets, N-queens

5.2 Templates worth memorising

```

from collections import deque, defaultdict
import heapq

# --- BFS on a grid (identical to flood-fill on a structured mesh) ---
def bfs_grid(grid, start):
    ny, nx = len(grid), len(grid[0])
    seen = {start}
    q = deque([(start, 0)])
    while q:
        (r, c), d = q.popleft()
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < ny and 0 <= nc < nx and (nr, nc) not in seen and grid[nr][nc] != 1:
                seen.add((nr, nc))
                q.append(((nr, nc), d + 1))
    return seen

# --- Union-Find with path compression + union by rank ---
class DSU:
    def __init__(self, n: int):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.components = n

    def find(self, x: int) -> int:
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]] # path halving
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) -> bool:
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.rank[ra] += self.rank[rb] == self.rank[rb]
        self.components -= 1
        return True

# --- Binary search on the answer (very common, often missed) ---
def min_feasible(lo: int, hi: int, feasible) -> int:
    """Smallest x in [lo, hi] with feasible(x) True, assuming monotone."""
    while lo < hi:
        mid = (lo + hi) // 2
        if feasible(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

# --- Dijkstra ---
def dijkstra(adj: dict[int, list[tuple[int, float]]], src: int) -> dict[int, float]:
    dist = {src: 0.0}
    pq = [(0.0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist.get(u, float("inf")):
            continue
        for v, w in adj[u]:
            nd = d + w
            if nd < dist.get(v, float("inf")):
                dist[v] = nd
                heapq.heappush(pq, (nd, v))
    return dist

# --- Sliding window (variable size) ---
def longest_unique(s: str) -> int:
    last, best, start = {}, 0, 0
    for i, ch in enumerate(s):
        if ch in last and last[ch] >= start:
            start = last[ch] + 1
        last[ch] = i
        best = max(best, i - start + 1)
    return best

```

5.3 Domain-flavoured DSA questions

These are the ones you are most likely to see in a CFD-ML interview, because the interviewer wants DSA and domain relevance:

Q5.1 Given a list of cells and their neighbours, count the number of disconnected fluid regions (connected components). *Union-Find or BFS.*

Q5.2 Given an unstructured mesh, renumber the nodes to minimise matrix bandwidth (Cuthill-McKee). *BFS from a peripheral node, ordering neighbours by degree.*

Q5.3 Given a set of probe timestamps and a sorted list of solver output times, for each probe find the two bracketing output times. *np.searchsorted or binary search.*

Q5.4 Given a stream of residual values, maintain the running median. *Two heaps.*

Q5.5 Partition a mesh graph into k roughly equal parts minimising cut edges. Explain the approach (you are not expected to implement METIS — describe recursive bisection / spectral partitioning using the Fiedler vector).

```
# Q5.2 - Reverse Cuthill-McKee, hand-rolled
from collections import deque

def rcm(adj: dict[int, list[int]]) -> list[int]:
    visited, order = set(), []
    for start in sorted(adj, key=lambda n: len(adj[n])): # lowest degree first
        if start in visited:
            continue
        q = deque([start]); visited.add(start)
        while q:
            u = q.popleft(); order.append(u)
            for v in sorted(adj[u], key=lambda n: len(adj[n])):
                if v not in visited:
                    visited.add(v); q.append(v)
    return order[::-1] # reverse -> better profile
```

Part 6 — Numerical Methods and CFD in Pure Python

This is your home turf. Interviewers use it to check that your CFD knowledge translates into code.

6.1 Finite differences and the model equations

```
import numpy as np

def central_2nd(phi, h, axis=0):
    return (np.roll(phi, -1, axis) - 2 * phi + np.roll(phi, 1, axis)) / h**2

def upwind_1st(phi, u, h, axis=0):
    """First-order upwind - unconditionally monotone, heavily diffusive."""
    fwd = (np.roll(phi, -1, axis) - phi) / h
    bwd = (phi - np.roll(phi, 1, axis)) / h
    return np.where(u > 0, bwd, fwd)
```

1-D linear convection — the “hello world” of CFD coding tests

```
def convect_1d(nx=201, nt=500, c=1.0, cfl=0.8, L=2.0):
    dx = L / (nx - 1)
    dt = cfl * dx / abs(c)
    x = np.linspace(0, L, nx)
    u = np.where((x >= 0.5) & (x <= 1.0), 2.0, 1.0)      # square wave
    for _ in range(nt):
        un = u.copy()                                   # essential - no aliasing
        u[1:] = un[1:] - c * dt / dx * (un[1:] - un[:-1]) # backward difference
        u[0] = un[0]                                    # inlet Dirichlet
    return x, u
```

They will ask **why** `un = u.copy()`. Without it you are using already-updated values inside the same sweep — that is Gauss-Seidel, not explicit Euler, and it changes the scheme.

1-D Burgers with viscosity — tests nonlinearity + diffusion

```
def burgers_1d(nx=401, nt=2000, nu=0.02, L=2 * np.pi, cfl=0.4):
    dx = L / (nx - 1)
    dt = min(cfl * dx / 2.0, 0.25 * dx**2 / nu)          # convective AND diffusive limits
    x = np.linspace(0, L, nx)
    u = np.sin(x) + 0.5
    for _ in range(nt):
        un = u.copy()
        conv = un * (un - np.roll(un, 1)) / dx           # upwind (u>0)
        diff = nu * (np.roll(un, -1) - 2*un + np.roll(un, 1)) / dx**2
        u = un + dt * (-conv + diff)
    return x, u
```

2-D lid-driven cavity — the classic take-home

```

def cavity(nx=41, ny=41, nt=500, nit=50, rho=1.0, nu=0.1, u_lid=1.0):
    dx, dy = 2.0 / (nx - 1), 2.0 / (ny - 1)
    dt = 0.001
    u = np.zeros((ny, nx)); v = np.zeros((ny, nx)); p = np.zeros((ny, nx))

    def build_rhs(u, v):
        b = np.zeros_like(u)
        b[1:-1, 1:-1] = rho * (
            1/dt * ((u[1:-1, 2:] - u[1:-1, :-2]) / (2*dx)
                    + (v[2:, 1:-1] - v[:-2, 1:-1]) / (2*dy))
            - ((u[1:-1, 2:] - u[1:-1, :-2]) / (2*dx))**2
            - 2 * ((u[2:, 1:-1] - u[:-2, 1:-1]) / (2*dy)
                  * (v[1:-1, 2:] - v[1:-1, :-2]) / (2*dx))
            - ((v[2:, 1:-1] - v[:-2, 1:-1]) / (2*dy))**2)
        return b

    for _ in range(nt):
        un, vn = u.copy(), v.copy()
        b = build_rhs(u, v)
        for _ in range(nit):
            # pressure Poisson, Jacobi
            pn = p.copy()
            pn[1:-1, 1:-1] = (((pn[1:-1, 2:] + pn[1:-1, :-2]) * dy**2
                               + (pn[2:, 1:-1] + pn[:-2, 1:-1]) * dx**2)
                               / (2 * (dx**2 + dy**2))
                               - dx**2 * dy**2 / (2 * (dx**2 + dy**2))) * b[1:-1, 1:-1])
            p[:, -1] = p[:, -2]; p[:, 0] = p[:, 1] # dp/dn = 0 on walls
            p[0, :] = p[1, :]; p[-1, :] = 0.0 # reference pressure at lid

            u[1:-1, 1:-1] = (un[1:-1, 1:-1]
                             - un[1:-1, 1:-1] * dt/dx * (un[1:-1, 1:-1] - un[1:-1, :-2])
                             - vn[1:-1, 1:-1] * dt/dy * (un[1:-1, 1:-1] - un[:-2, 1:-1])
                             - dt / (2*rho*dx) * (p[1:-1, 2:] - p[1:-1, :-2])
                             + nu * (dt/dx**2 * (un[1:-1, 2:] - 2*un[1:-1, 1:-1] + un[1:-1, :-2])
                                     + dt/dy**2 * (un[2:, 1:-1] - 2*un[1:-1, 1:-1] + un[:-2, 1:-1])))
            v[1:-1, 1:-1] = (vn[1:-1, 1:-1]
                             - un[1:-1, 1:-1] * dt/dx * (vn[1:-1, 1:-1] - vn[1:-1, :-2])
                             - vn[1:-1, 1:-1] * dt/dy * (vn[1:-1, 1:-1] - vn[:-2, 1:-1])
                             - dt / (2*rho*dy) * (p[2:, 1:-1] - p[:-2, 1:-1])
                             + nu * (dt/dx**2 * (vn[1:-1, 2:] - 2*vn[1:-1, 1:-1] + vn[1:-1, :-2])
                                     + dt/dy**2 * (vn[2:, 1:-1] - 2*vn[1:-1, 1:-1] + vn[:-2, 1:-1])))

            u[0, :] = 0; u[:, 0] = 0; u[:, -1] = 0; u[-1, :] = u_lid
            v[0, :] = 0; v[-1, :] = 0; v[:, 0] = 0; v[:, -1] = 0
        return u, v, p

```

6.2 Stability: the numbers you must recite

Constraint	Formula	Meaning
CFL (convection, explicit)	$C = u \, dt / dx \leq 1$	Information cannot cross more than one cell per step
Diffusion number	$d = nu \, dt / dx^2 \leq 0.5$ (1-D)	Explicit diffusion limit; ≤ 0.25 in 2-D, $1/6$ in 3-D
Combined	$dt = \min(C \, dx / u , d \, dx^2 / nu)$	Take the stricter one
Peclet (cell)	$Pe = u \, dx / nu$	$Pe > 2$ with central differencing gives oscillations
Courant for acoustics	$(u + c) \, dt / dx \leq 1$	Compressible explicit schemes

```

def stable_dt(u_max: float, dx: float, nu: float,
              cfl: float = 0.5, dnum: float = 0.25) -> float:
    dt_conv = cfl * dx / max(u_max, 1e-12)
    dt_diff = dnum * dx * dx / max(nu, 1e-12)
    return min(dt_conv, dt_diff)

```

6.3 Verification: the habit that impresses

Always demonstrate the **observed order of accuracy**. This is a five-line addition that transforms a mediocre take-home into a strong one.


```

def observed_order(errors: list[float], refinements: list[float]) -> list[float]:
    """p = log(e1/e2) / log(h1/h2)."""
    return [np.log(errors[i] / errors[i+1]) / np.log(refinements[i] / refinements[i+1])
            for i in range(len(errors) - 1)]

def mms_check():
    """Method of Manufactured Solutions: choose phi, derive the source term analytically."""
    # phi = sin(pi x) sin(pi y) -> -laplacian(phi) = 2 pi^2 sin(pi x) sin(pi y)
    for n in (17, 33, 65, 129):
        h = 1.0 / (n - 1)
        x = y = np.linspace(0, 1, n)
        X, Y = np.meshgrid(x, y, indexing="ij")
        exact = np.sin(np.pi * X) * np.sin(np.pi * Y)
        f = 2 * np.pi**2 * exact
        num = poisson_solve(f, h)
        err = np.sqrt(np.mean((num - exact) ** 2)) # L2 norm
        print(f"n={n:4d}  h={h:.4e}  L2={err:.4e}")

```

Expect the L2 error to fall by a factor of 4 each time you halve `h` for a second-order scheme.

6.4 Practice set — Part 6

Q6.1 Implement 1-D advection with (a) first-order upwind, (b) Lax-Wendroff, (c) a MUSCL scheme with the minmod limiter. Compare on a square wave and explain the trade-off between numerical diffusion and dispersive oscillations (Godunov's theorem).

Q6.2 Solve the 2-D heat equation with explicit Euler and with Crank-Nicolson (sparse). Show that the explicit version blows up when $\nu \, dt/dx^2 > 0.25$, and that the implicit one does not.

Q6.3 Write a general `stable_dt` for a non-uniform 3-D grid given cell volumes and face areas.

Q6.4 Given a velocity field on a structured grid, compute vorticity, Q-criterion, and the divergence; report the maximum divergence as a mass-conservation check.

Q6.5 Implement the method of manufactured solutions for the 1-D convection-diffusion equation and confirm the design order of your discretisation.

Part 7 — Meshes, Geometry and Unstructured Data

Anything involving MeshGraphNets requires you to be comfortable turning a mesh into a graph. This part is the bridge between CFD data and ML input.

7.1 Reading meshes

```
import meshio
import numpy as np

mesh = meshio.read("case.vtu")
points = mesh.points                # (n_nodes, 3)
tris = mesh.cells_dict.get("triangle") # (n_cells, 3)
tets = mesh.cells_dict.get("tetra")   # (n_cells, 4)
u = mesh.point_data["U"]             # (n_nodes, 3)
p_cell = mesh.cell_data["p"][0]      # per cell block

meshio.write("out.vtu", meshio.Mesh(points, [{"triangle", tris}],
                                     point_data={"pred": pred}))
```

```
import pyvista as pv

grid = pv.read("case.vtu")
grid = grid.cell_data_to_point_data() # cell -> node interpolation
slice_ = grid.slice(normal="z", origin=(0, 0, 0.5))
surf = grid.extract_surface()
grid = grid.compute_derivative(scalars="U", gradient=True, qcriterion=True)
probe = grid.sample_over_line((0, 0, 0), (1, 0, 0), resolution=200)
```

7.2 Mesh to graph — the MeshGraphNet preprocessing step

```
import numpy as np

def edges_from_triangles(tris: np.ndarray) -> np.ndarray:
    """Return unique undirected edges as a (2, n_edges) index array, both directions."""
    e = np.concatenate([tris[:, [0, 1]], tris[:, [1, 2]], tris[:, [2, 0]]], axis=0)
    e = np.sort(e, axis=1) # canonical orientation
    e = np.unique(e, axis=0) # dedup shared edges
    both = np.concatenate([e, e[:, ::-1]], axis=0) # make it bidirectional
    return both.T # (2, E) - PyG convention

def edge_features(pos: np.ndarray, edge_index: np.ndarray) -> np.ndarray:
    """MeshGraphNet edge features: relative displacement + its norm."""
    src, dst = edge_index
    rel = pos[src] - pos[dst] # (E, dim)
    dist = np.linalg.norm(rel, axis=1, keepdims=True)
    return np.concatenate([rel, dist], axis=1) # (E, dim+1)

def world_edges(pos: np.ndarray, radius: float, edge_index: np.ndarray) -> np.ndarray:
    """Extra 'world-space' edges for contact/collision, as in the original paper."""
    from scipy.spatial import cKDTree
    pairs = cKDTree(pos).query_pairs(radius, output_type="ndarray")
    pairs = np.concatenate([pairs, pairs[:, ::-1]], axis=0).T
    existing = set(map(tuple, edge_index.T))
    keep = [i for i in range(pairs.shape[1]) if tuple(pairs[:, i]) not in existing]
    return pairs[:, keep]
```

Concept	In MeshGraphNet terms
Node	Mesh vertex
Node features	Velocity, pressure, node type one-hot (fluid / wall / inlet / outlet)
Mesh edge	Vertex pair sharing a mesh element
Mesh edge features	Relative position in mesh space and its norm
World edge	Proximity pair within a radius in world space (deforming/contact problems)
Output	Per-node time derivative or increment, integrated forward

7.3 Geometric quantities

```
def triangle_areas_normals(p: np.ndarray, tris: np.ndarray):
    a, b, c = p[tris[:, 0]], p[tris[:, 1]], p[tris[:, 2]]
    cross = np.cross(b - a, c - a)
    area = 0.5 * np.linalg.norm(cross, axis=1)
    normal = cross / np.maximum(np.linalg.norm(cross, axis=1, keepdims=True), 1e-30)
    return area, normal

def tet_volumes(p: np.ndarray, tets: np.ndarray) -> np.ndarray:
    a, b, c, d = (p[tets[:, i]] for i in range(4))
    return np.abs(np.einsum("ij,ij->i", b - a, np.cross(c - a, d - a))) / 6.0

def cell_to_node(cell_values, node_to_cells_idx, weights=None):
    """Volume-weighted cell-to-node interpolation via bincount scatter."""
    src, dst = node_to_cells_idx # (n_pairs,) cell ids and node ids
    w = np.ones_like(src, dtype=float) if weights is None else weights
    num = np.bincount(dst, weights=w * cell_values[src], minlength=n_nodes)
    den = np.bincount(dst, weights=w, minlength=n_nodes)
    return num / np.maximum(den, 1e-30)
```

7.4 Signed distance and wall distance

```
from scipy.spatial import cKDTree

def wall_distance(cell_centres: np.ndarray, wall_points: np.ndarray) -> np.ndarray:
    """Nearest-wall distance - a standard feature for turbulence-model ML."""
    return cKDTree(wall_points).query(cell_centres, k=1)[0]

def signed_distance(pts, surf_pts, surf_normals):
    tree = cKDTree(surf_pts)
    d, idx = tree.query(pts, k=1)
    sign = np.sign(np.einsum("ij,ij->i", pts - surf_pts[idx], surf_normals[idx]))
    return sign * d
```

7.5 Practice set — Part 7

Q7.1 Given `points` and `tets`, build a bidirectional edge list with no duplicates, and report the mean node degree. Vectorise fully.

Q7.2 Compute per-node normals for a triangulated surface by area-weighted averaging of face normals.

Q7.3 Given two meshes of the same geometry at different resolutions, map a field from the fine to the coarse mesh conservatively (preserving the volume integral), and quantify the error.

Q7.4 Implement a node-type one-hot encoder given a dict of boundary patch names to node ids, including a default "interior" class.

Part 8 — PyTorch Fundamentals

Everything in Parts 9 and 10 rests on this. Assume live coding without documentation.

8.1 Tensors and devices

```
import torch

x = torch.randn(1000, 3, dtype=torch.float32)
device = "cuda" if torch.cuda.is_available() else "cpu"
x = x.to(device)

x.shape, x.dtype, x.device, x.requires_grad
y = x.view(-1)          # needs contiguity
y = x.reshape(-1)       # copies if needed - safer
z = x.permute(1, 0)     # view, non-contiguous afterwards
z = z.contiguous()

torch.from_numpy(arr)   # SHARES memory with the NumPy array
t.numpy()               # shares memory; fails if requires_grad -> use .detach().numpy()
t.cpu().detach().numpy()
```

NumPy	PyTorch
<code>np.concatenate</code>	<code>torch.cat</code>
<code>np.stack</code>	<code>torch.stack</code>
<code>arr[:, None]</code>	<code>t.unsqueeze(1)</code>
<code>np.einsum</code>	<code>torch.einsum</code>
<code>np.clip</code>	<code>torch.clamp</code>
<code>np.where</code>	<code>torch.where</code>
<code>np.bincount(w=...)</code>	<code>torch.zeros(n).index_add_(0, idx, vals)</code> or <code>scatter_add_</code>

8.2 Autograd — the single most important topic for PINNs

```
x = torch.linspace(0, 1, 100, requires_grad=True).reshape(-1, 1)
y = x ** 3

dy_dx = torch.autograd.grad(
    outputs=y, inputs=x,
    grad_outputs=torch.ones_like(y),
    create_graph=True,      # REQUIRED to differentiate again (second derivative)
    retain_graph=True,
)[0]

d2y_dx2 = torch.autograd.grad(dy_dx, x, torch.ones_like(dy_dx), create_graph=True)[0]
# d2y/dx2 = 6x
```

```
def gradient(y: torch.Tensor, x: torch.Tensor) -> torch.Tensor:
    """Reusable helper - write this from memory in an interview."""
    return torch.autograd.grad(y, x, grad_outputs=torch.ones_like(y),
                               create_graph=True, retain_graph=True)[0]

def divergence(vec: torch.Tensor, x: torch.Tensor) -> torch.Tensor:
    """div(u) for vec of shape (N, dim) and x of shape (N, dim)."""
    return sum(gradient(vec[:, i:i+1], x)[:, i:i+1] for i in range(vec.shape[1]))

def laplacian(y: torch.Tensor, x: torch.Tensor) -> torch.Tensor:
    g = gradient(y, x)
    return sum(gradient(g[:, i:i+1], x)[:, i:i+1] for i in range(x.shape[1]))
```

Guaranteed question: “Why `create_graph=True`?” Because the derivative itself must remain a differentiable function of the parameters, so that the PDE residual can be backpropagated into the network weights. With `create_graph=False` you get a detached tensor and the physics loss contributes no gradient.

Second guaranteed question: “What is the difference between `detach()`, `no_grad()` and `requires_grad_(False)`?” `detach()` returns a tensor sharing storage but cut from the graph; `torch.no_grad()` is a context that stops graph recording entirely (use for inference/eval); `requires_grad_(False)` freezes a parameter so the optimiser produces no gradient for it.

8.3 Models, losses, optimisers

```
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, in_dim, out_dim, hidden=128, layers=6, act=nn.Tanh):
        super().__init__()
        dims = [in_dim] + [hidden] * layers + [out_dim]
        seq = []
        for i in range(len(dims) - 1):
            seq.append(nn.Linear(dims[i], dims[i + 1]))
            if i < len(dims) - 2:
                seq.append(act())
        self.net = nn.Sequential(*seq)
        self.apply(self._init)

    @staticmethod
    def _init(m):
        if isinstance(m, nn.Linear):
            nn.init.xavier_normal_(m.weight)      # Xavier suits tanh; He suits ReLU
            nn.init.zeros_(m.bias)

    def forward(self, x):
        return self.net(x)
```

```
model = MLP(3, 4).to(device)
opt = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=0.0)
sched = torch.optim.lr_scheduler.ReduceLROnPlateau(opt, factor=0.5, patience=200)

for epoch in range(n_epochs):
    model.train()
    opt.zero_grad(set_to_none=True)      # set_to_none is faster than zeroing
    pred = model(xb)
    loss = nn.functional.mse_loss(pred, yb)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    opt.step()
    sched.step(loss)

    if epoch % 100 == 0:
        model.eval()
        with torch.no_grad():
            val = nn.functional.mse_loss(model(xv), yv)
        print(f"epoch:6d} train {loss.item():.3e} val {val.item():.3e}")
```

Optimiser	When
Adam	Default for almost everything; PINNs, GNNs
AdamW	Adam with decoupled weight decay — correct L2 regularisation
L-BFGS	PINN fine-tuning after Adam; full-batch only, needs a closure
SGD + momentum	Large-scale vision; rarely for scientific ML

```
# L-BFGS - the PINN second-stage optimiser. Note the closure signature.
lbfgs = torch.optim.LBFGS(model.parameters(), lr=1.0, max_iter=5000,
                           history_size=50, tolerance_grad=1e-9,
                           line_search_fn="strong_wolfe")

def closure():
    lbfgs.zero_grad()
    loss = total_loss()
    loss.backward()
    return loss

lbfgs.step(closure)
```

8.4 Datasets and DataLoaders

```
from torch.utils.data import Dataset, DataLoader, random_split

class CFDSnapshotDataset(Dataset):
    def __init__(self, h5_path: str, keys: list[str]):
        self.h5_path, self.keys = h5_path, keys
        self._f = None  # open lazily -> multiprocessing safe

    def __len__(self):
        return len(self.keys)

    def __getitem__(self, i):
        import h5py
        if self._f is None:
            self._f = h5py.File(self.h5_path, "r")
        g = self._f[self.keys[i]]
        x = torch.from_numpy(g["coords"][:]).float()
        y = torch.from_numpy(g["U"][:]).float()
        return x, y

loader = DataLoader(ds, batch_size=16, shuffle=True, num_workers=4,
                    pin_memory=True, drop_last=True, persistent_workers=True)
```

Trap they check for: opening an HDF5 handle in `__init__` breaks with `num_workers > 0` because the handle cannot be pickled/forked safely. Open it lazily inside `__getitem__`.

8.5 Practice set — Part 8

Q8.1 Write a function that computes the Jacobian of a network output with respect to its input for a batch, using `torch.autograd.functional.jacobian` and then again manually with a loop over output components. Compare the cost.

Q8.2 Implement a custom `nn.Module` for a Fourier feature embedding `[sin(2πBx), cos(2πBx)]` with a fixed random Gaussian `B`, and explain why it helps PINNs.

Q8.3 Implement gradient accumulation to simulate a batch size of 256 with only enough memory for 32.

Q8.4 Write a training loop with early stopping, best-checkpoint saving, and full reproducibility (seeding all RNGs, deterministic algorithms).

Q8.5 Debug: a model trains to a low loss but predicts a constant value. List five plausible causes.

Answers to Q8.5: (1) the target was normalised to near-zero variance; (2) learning rate too high so weights collapsed; (3) dead ReLUs / saturated tanh from bad initialisation or unscaled inputs; (4) the loss is dominated by a term that is minimised by the mean (e.g. MSE with an uninformative input); (5) the input features carry no signal after a preprocessing bug — e.g. normalising by the wrong axis.

Part 9 — Physics-Informed Neural Networks (PINNs)

9.1 The formulation

For a PDE $N[u](x, t) = 0$ on domain Ω with boundary/initial conditions, a PINN approximates $u \approx u_\theta(x, t)$ and minimises

$$L(\theta) = \lambda_f * L_{pde} + \lambda_b * L_{bc} + \lambda_\theta * L_{ic} + \lambda_d * L_{data}$$
$$L_{pde} = (1/N_f) \sum |N[u_\theta](x_f, t_f)|^2 \quad \text{at collocation points}$$
$$L_{bc} = (1/N_b) \sum |B[u_\theta](x_b, t_b)|^2$$
$$L_{ic} = (1/N_\theta) \sum |u_\theta(x, 0) - u_\theta(x)|^2$$
$$L_d = (1/N_d) \sum |u_\theta(x_d, t_d) - u_{obs}|^2$$

Derivatives in $N[\cdot]$ come from automatic differentiation, not from a mesh. That is the entire idea, and it is why Section 8.2 matters so much.

9.2 Complete PINN: 1-D Burgers equation

This is *the* canonical PINN benchmark (Raissi et al.). Be able to write it in ~40 lines.

```
import numpy as np
import torch
import torch.nn as nn

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
torch.manual_seed(0); np.random.seed(0)

class PINN(nn.Module):
    def __init__(self, layers=(2, 64, 64, 64, 64, 1)):
        super().__init__()
        net = []
        for i in range(len(layers) - 1):
            net.append(nn.Linear(layers[i], layers[i + 1]))
            if i < len(layers) - 2:
                net.append(nn.Tanh())
        self.net = nn.Sequential(*net)
        for m in self.net:
            if isinstance(m, nn.Linear):
                nn.init.xavier_normal_(m.weight); nn.init.zeros_(m.bias)

    def forward(self, x, t):
        return self.net(torch.cat([x, t], dim=1))

    def grad(y, x):
        return torch.autograd.grad(y, x, torch.ones_like(y),
                                    create_graph=True, retain_graph=True)[0]

    def pde_residual(model, x, t, nu=0.01 / np.pi):
        u = model(x, t)
        u_t = grad(u, t)
        u_x = grad(u, x)
        u_xx = grad(u_x, x)
        return u_t + u * u_x - nu * u_xx

    def train_burgers(n_f=10_000, n_b=200, n_\theta=200, epochs=20_000):
        model = PINN().to(DEVICE)

        # Collocation points - Latin hypercube would be better, uniform is fine
        x_f = torch.rand(n_f, 1, device=DEVICE) * 2 - 1 # x in [-1, 1]
        t_f = torch.rand(n_f, 1, device=DEVICE) # t in [0, 1]
        x_f.requires_grad_(True); t_f.requires_grad_(True)

        # Initial condition u(x, 0) = -sin(pi x)
        x_\theta = torch.linspace(-1, 1, n_\theta, device=DEVICE).reshape(-1, 1)
        t_\theta = torch.zeros_like(x_\theta)
        u_\theta = -torch.sin(np.pi * x_\theta)

        # Boundary condition u(-1, t) = u(1, t) = 0
        t_b = torch.rand(n_b, 1, device=DEVICE)
```

```

x_b = torch.cat([-torch.ones(n_b // 2, 1), torch.ones(n_b - n_b // 2, 1)]).to(DEVICE)
u_b = torch.zeros_like(t_b)

opt = torch.optim.Adam(model.parameters(), lr=1e-3)
sched = torch.optim.lr_scheduler.StepLR(opt, step_size=5000, gamma=0.5)
mse = nn.MSELoss()

for ep in range(epochs):
    opt.zero_grad(set_to_none=True)
    l_pde = mse(pde_residual(model, x_f, t_f), torch.zeros(n_f, 1, device=DEVICE))
    l_ic = mse(model(x_0, t_0), u_0)
    l_bc = mse(model(x_b, t_b), u_b)
    loss = l_pde + 10.0 * l_ic + 10.0 * l_bc
    loss.backward()
    opt.step(); sched.step()
    if ep % 1000 == 0:
        print(f"ep:6d} pde {l_pde.item():.2e} ic {l_ic.item():.2e} "
              f"bc {l_bc.item():.2e}")

# Second stage: L-BFGS polish
lbfgs = torch.optim.LBFGS(model.parameters(), lr=1.0, max_iter=5000,
                           line_search_fn="strong_wolfe")

def closure():
    lbfgs.zero_grad()
    l = (mse(pde_residual(model, x_f, t_f), torch.zeros(n_f, 1, device=DEVICE))
         + 10.0 * mse(model(x_0, t_0), u_0)
         + 10.0 * mse(model(x_b, t_b), u_b))
    l.backward()
    return l

lbfgs.step(closure)
return model

```

9.3 PINN for steady 2-D Navier-Stokes with a stream function

Using a stream function ψ with $u = \partial\psi/\partial y$, $v = -\partial\psi/\partial x$ enforces incompressibility **exactly**, removing the continuity loss term. Mentioning this in an interview is a strong signal.

```

def ns_residual_streamfunction(model, x, y, rho=1.0, nu=0.01):
    out = model(torch.cat([x, y], dim=1))
    psi, p = out[:, 0:1], out[:, 1:2]

    u = grad(psi, y)
    v = -grad(psi, x)

    u_x, u_y = grad(u, x), grad(u, y)
    v_x, v_y = grad(v, x), grad(v, y)
    u_xx, u_yy = grad(u_x, x), grad(u_y, y)
    v_xx, v_yy = grad(v_x, x), grad(v_y, y)
    p_x, p_y = grad(p, x), grad(p, y)

    f_u = rho * (u * u_x + v * u_y) + p_x - rho * nu * (u_xx + u_yy)
    f_v = rho * (u * v_x + v * v_y) + p_y - rho * nu * (v_xx + v_yy)
    return f_u, f_v, u, v, p # continuity satisfied identically

```

9.4 Hard constraints — the trick that fixes most convergence problems

Rather than penalising boundary conditions, build them into the ansatz:

```

def hard_bc_output(model, x, t):
    """u(x,t) = (1 - x^2) * NN + (1-t) * u0(x) -> BC and IC satisfied exactly."""
    nn_out = model(x, t)
    u0 = -torch.sin(np.pi * x)
    return (1.0 - x**2) * t * nn_out + (1.0 - t) * u0

```


Approach	Pros	Cons
Soft (penalty)	General, easy	Needs weight tuning; BCs only approximately satisfied
Hard (ansatz)	BCs exact, faster convergence, fewer loss terms	Requires an analytic distance/lift function; hard for complex geometry
Approximate distance functions (R-functions)	Hard BCs on complex domains	Extra machinery

9.5 Loss balancing — the number-one PINN failure mode

```

class LearnableWeights(nn.Module):
    """Uncertainty weighting (Kendall):  $L = \sum(\exp(-s_i) L_i + s_i)$ ."""
    def __init__(self, n_terms: int):
        super().__init__()
        self.log_sigma = nn.Parameter(torch.zeros(n_terms))

    def forward(self, losses: list[torch.Tensor]) -> torch.Tensor:
        return sum(torch.exp(-self.log_sigma[i]) * l + self.log_sigma[i]
                    for i, l in enumerate(losses))

def grad_norm_weights(losses, shared_params, alpha=0.9, old_w=None):
    """Learning-rate annealing (Wang, Teng & Perdikaris 2021):
    scale each loss so that its gradient magnitude matches the PDE term."""
    norms = []
    for l in losses:
        g = torch.autograd.grad(l, shared_params, retain_graph=True, allow_unused=True)
        norms.append(torch.sqrt(sum((gi ** 2).sum() for gi in g if gi is not None)))
    ref = norms[0]
    new_w = [float(ref / (n + 1e-12)) for n in norms]
    if old_w is not None:
        new_w = [alpha * o + (1 - alpha) * n for o, n in zip(old_w, new_w)]
    return new_w

```

9.6 Adaptive sampling

```

def rar_sample(model, residual_fn, n_pool=100_000, n_add=1000, domain=(-1, 1, 0, 1)):
    """Residual-based Adaptive Refinement: add points where the residual is largest."""
    x = torch.rand(n_pool, 1) * (domain[1] - domain[0]) + domain[0]
    t = torch.rand(n_pool, 1) * (domain[3] - domain[2]) + domain[2]
    x.requires_grad_(True); t.requires_grad_(True)
    with torch.enable_grad():
        r = residual_fn(model, x, t).abs().detach().squeeze()
    idx = torch.topk(r, n_add).indices
    return x[idx].detach(), t[idx].detach()

```

9.7 The PINN failure catalogue — expect to be asked

Symptom	Cause	Fix
Loss plateaus at the trivial solution $u \equiv 0$	PDE residual is minimised by a trivial field; BC/IC weight too low	Raise BC/IC weights, use hard constraints, check the PDE is not homogeneous with trivial solution
Solution accurate near $t=0$, garbage later	Causality violation — the network fits the easy early region	Causal weighting, time-marching in windows, curriculum on t
Cannot capture the shock in Burgers	Spectral bias — MLPs learn low frequencies first	Fourier features, adaptive activation, residual-adaptive sampling, domain decomposition (XPINN/cPINN)
Training unstable at high Re	Stiff residual, badly scaled loss terms	Non-dimensionalise, gradient-norm loss balancing, curriculum on Re
Good training loss, wrong physics	Under-constrained inverse problem	Add data, add conservation constraints, regularise the unknown parameters
Slow, memory-heavy	Second derivatives via autograd cost $O(\text{dim})$ extra backward passes	Fewer collocation points per step with resampling; first-order (mixed/VPINN) formulations

Non-dimensionalisation is the answer to a surprising number of PINN questions. If x is in metres over a 3 m car and u is 30 m/s and p is 10^5 Pa, the loss terms differ by ten orders of magnitude and no weighting scheme will save you. Scale first.

9.8 PINNs vs classical solvers — the judgement question

You will be asked: “When would you actually use a PINN instead of a CFD solver?” A defensible answer:

- **Good fit:** inverse problems and parameter identification; data assimilation where you have sparse sensor data and partial physics; problems with unknown boundary conditions; high-dimensional parametric PDEs where meshing is the bottleneck; super-resolution or denoising of measured fields with physics as a regulariser.
- **Poor fit:** as a replacement for a well-validated forward solver on a fixed geometry. A finite-volume solver is faster and more accurate for a single deterministic run. PINNs do not currently beat classical methods on forward high-Reynolds turbulent flow.
- **The honest framing:** PINNs turn a PDE into an optimisation problem; you trade guaranteed discretisation error for unbounded optimisation error. Say that and you sound like someone who has actually used them.

9.9 Practice set — Part 9

Q9.1 Write a PINN for the 1-D heat equation $u_t = \alpha u_{xx}$ with $u(0,t)=u(1,t)=0$ and $u(x,0)=\sin(\pi x)$. Compare against the analytic solution $e^{-\alpha\pi^2 t} \sin(\pi x)$ and report the relative L2 error.

Q9.2 Modify it to use a hard-constraint ansatz so no BC/IC loss terms are needed. Compare convergence.

Q9.3 Solve an **inverse** problem: given noisy measurements of u , recover the unknown diffusivity α by making it a `nn.Parameter`. Report the identified value and its sensitivity to noise level.

Q9.4 Implement the 2-D steady lid-driven cavity as a PINN using the stream-function formulation. Compare centreline velocity profiles against Ghia et al. benchmark values.

Q9.5 Implement Fourier feature embeddings and show quantitatively that they improve the fit of a high-frequency solution $u = \sin(20\pi x)$.

Q9.6 Implement residual-based adaptive refinement and plot where the added collocation points cluster for the Burgers problem. *Expected: at the shock near $x=0$.*

```
# Q9.3 - inverse PINN skeleton
class InversePINN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = MLP(2, 1, hidden=64, layers=5)
        self.log_alpha = nn.Parameter(torch.tensor(0.0)) # learn log for positivity

    @property
    def alpha(self):
        return torch.exp(self.log_alpha)

    def forward(self, x, t):
        return self.net(torch.cat([x, t], 1))

    def inverse_residual(model, x, t):
        u = model(x, t)
        return grad(u, t) - model.alpha * grad(grad(u, x), x)

# loss = mse(residual, 0) + lambda_d * mse(model(x_d, t_d), u_measured)
# Optimise network weights AND log_alpha jointly with one Adam instance.
```

Part 10 — Graph Neural Networks and MeshGraphNets

10.1 Why graphs for CFD

A CFD mesh *is* a graph: cells/nodes are vertices, face adjacency gives edges. A GNN respects this structure and is therefore mesh-resolution-flexible, permutation-equivariant, and applies to unstructured geometries where a CNN cannot. This is the honest one-paragraph pitch and you should be able to give it.

Property	CNN	GNN
Domain	Regular grid	Arbitrary graph
Weight sharing	Translation	Permutation-equivariant across nodes/edges
Unstructured mesh	Needs interpolation to a grid	Native
Resolution transfer	Poor	Reasonable — the same MLPs act on local features
Long-range coupling	Deep stacks or dilation	Message-passing steps, or multigrid/hierarchical variants

10.2 Message passing from scratch (no PyG) — write this in an interview

```
import torch
import torch.nn as nn

def build_mlp(in_dim, hidden, out_dim, layers=2, layer_norm=True):
    dims = [in_dim] + [hidden] * layers + [out_dim]
    mods = []
    for i in range(len(dims) - 1):
        mods.append(nn.Linear(dims[i], dims[i + 1]))
        if i < len(dims) - 2:
            mods.append(nn.ReLU())
    if layer_norm:
        mods.append(nn.LayerNorm(out_dim))
    return nn.Sequential(*mods)

class MessagePassingLayer(nn.Module):
    """One GraphNet block: edge update, aggregation, node update, residual."""

    def __init__(self, node_dim, edge_dim, hidden=128):
        super().__init__()
        self.edge_mlp = build_mlp(2 * node_dim + edge_dim, hidden, edge_dim)
        self.node_mlp = build_mlp(node_dim + edge_dim, hidden, node_dim)

    def forward(self, node_feat, edge_feat, edge_index):
        src, dst = edge_index # (E, ), (E, )

        # 1. Edge update: e'_{ij} = f_e([v_i, v_j, e_{ij}])
        edge_in = torch.cat([node_feat[src], node_feat[dst], edge_feat], dim=-1)
        edge_out = self.edge_mlp(edge_in)

        # 2. Aggregate to receiver nodes (sum is permutation-invariant and
        #    preserves the flux-like semantics of a conservation law)
        agg = torch.zeros_like(node_feat)
        agg.index_add_(0, dst, edge_out)

        # 3. Node update: v'_i = f_v([v_i, sum_j e'_{ij}])
        node_out = self.node_mlp(torch.cat([node_feat, agg], dim=-1))

        # 4. Residual connections - essential for stacking 10-15 layers
        return node_feat + node_out, edge_feat + edge_out
```

Aggregation choice question. `sum` preserves magnitude information and mirrors a flux balance over faces — the right default for physics. `mean` is invariant to node degree, useful when mesh density varies wildly. `max` picks out extremes and is common in point clouds. Say that `sum` is the MeshGraphNet choice and explain why.

10.3 MeshGraphNet: encode-process-decode

```
class MeshGraphNet(nn.Module):
    """Pfaff et al. 2021, 'Learning Mesh-Based Simulation with Graph Networks'."""

    def __init__(self, node_in, edge_in, node_out, hidden=128, n_blocks=15):
        super().__init__()
        self.node_encoder = build_mlp(node_in, hidden, hidden)
        self.edge_encoder = build_mlp(edge_in, hidden, hidden)
        self.blocks = nn.ModuleList(
            MessagePassingLayer(hidden, hidden, hidden) for _ in range(n_blocks))
        self.decoder = build_mlp(hidden, hidden, node_out, layer_norm=False)

    def forward(self, node_feat, edge_feat, edge_index):
        v = self.node_encoder(node_feat)
        e = self.edge_encoder(edge_feat)
        for block in self.blocks:
            v, e = block(v, e, edge_index)
        return self.decoder(v)  # per-node output: du/dt or increment
```

Feature construction (this is where marks are won or lost)

```
NODE_TYPES = {"fluid": 0, "wall": 1, "inlet": 2, "outlet": 3, "obstacle": 4}

def make_features(pos, vel, node_type, edge_index, n_types=len(NODE_TYPES)):
    """pos: (N, d) mesh coords; vel: (N, d); node_type: (N,) int."""
    one_hot = torch.nn.functional.one_hot(node_type, n_types).float()
    node_feat = torch.cat([vel, one_hot], dim=-1)  # NOTE: no absolute position

    src, dst = edge_index
    rel = pos[src] - pos[dst]
    edge_feat = torch.cat([rel, rel.norm(dim=-1, keepdim=True)], dim=-1)
    return node_feat, edge_feat
```

Why is absolute position excluded from node features? To make the model **translation-invariant**. All spatial information enters through *relative* displacements on edges. Including absolute coordinates lets the network memorise the specific domain and destroys generalisation to a shifted or new geometry. This is a standard interview question and a standard candidate mistake.

Normalisation with running statistics

```

class Normalizer(nn.Module):
    """Online mean/std accumulation, as in the DeepMind reference implementation."""

    def __init__(self, size, max_accum=1e6, eps=1e-8):
        super().__init__()
        self.register_buffer("count", torch.zeros(1))
        self.register_buffer("sum", torch.zeros(size))
        self.register_buffer("sum_sq", torch.zeros(size))
        self.max_accum, self.eps = max_accum, eps

    def forward(self, x, accumulate=True):
        if accumulate and self.training and self.count < self.max_accum:
            self.count += x.shape[0]
            self.sum += x.sum(0)
            self.sum_sq += (x ** 2).sum(0)
            return (x - self._mean()) / self._std()

    def inverse(self, x):
        return x * self._std() + self._mean()

    def _mean(self):
        return self.sum / self.count.clamp(min=1.0)

    def _std(self):
        var = self.sum_sq / self.count.clamp(min=1.0) - self._mean() ** 2
        return var.clamp(min=self.eps ** 2).sqrt()

```

Training-noise injection — the key stability trick

```

def training_step(model, batch, noise_std=2e-2):
    """Add noise to inputs, but correct the target so the model learns to
    RETURN TO the true trajectory. Without this, rollouts diverge."""
    vel = batch.velocity
    mask = (batch.node_type == NODE_TYPES["fluid"]).unsqueeze(-1)

    noise = torch.randn_like(vel) * noise_std * mask
    noisy_vel = vel + noise

    target = batch.next_velocity - vel          # true increment
    adjusted_target = target - noise            # <- the correction

    pred = model(*make_features(batch.pos, noisy_vel, batch.node_type,
                                batch.edge_index), batch.edge_index)
    return ((pred - adjusted_target) ** 2)[mask.squeeze(-1)].mean()

```

Explain the noise trick. A one-step-trained autoregressive model sees only perfect inputs during training but its own imperfect outputs during rollout. The distribution shift compounds and the rollout explodes. Injecting noise of a magnitude comparable to the expected one-step error, while adjusting the target so the network still points at the ground truth, teaches an error-correcting map and makes rollouts of hundreds of steps stable. This is the single most important implementation detail in the paper, and being able to state it clearly is a strong differentiator.

Autoregressive rollout

```

@torch.no_grad()
def rollout(model, initial_state, n_steps, dt=1.0):
    state = initial_state.clone()
    traj = [state.clone()]
    for _ in range(n_steps):
        node_feat, edge_feat = make_features(pos, state, node_type, edge_index)
        delta = model(node_feat, edge_feat, edge_index)
        state = state + delta * dt
        state[boundary_mask] = prescribed[boundary_mask] # re-impose hard BCs each step
        traj.append(state.clone())
    return torch.stack(traj)

```

10.4 The same thing with PyTorch Geometric

```

import torch
from torch_geometric.data import Data, Batch
from torch_geometric.loader import DataLoader
from torch_geometric.nn import MessagePassing, GCNConv, GATConv
from torch_geometric.utils import scatter

data = Data(x=node_feat,          # (N, F_node)
            edge_index=edge_index, # (2, E) long
            edge_attr=edge_feat,   # (E, F_edge)
            y=target,              # (N, F_out)
            pos=pos)

loader = DataLoader(dataset, batch_size=8, shuffle=True)
# PyG batches graphs by block-diagonal concatenation; `batch.batch` maps node -> graph id

class MGNLayer(MessagePassing):
    def __init__(self, hidden):
        super().__init__(aggr="add") # aggregation declared here
        self.edge_mlp = build_mlp(3 * hidden, hidden, hidden)
        self.node_mlp = build_mlp(2 * hidden, hidden, hidden)

    def forward(self, x, edge_index, edge_attr):
        out, new_edge = self.propagate(edge_index, x=x, edge_attr=edge_attr), None
        return x + out, edge_attr

    def message(self, x_i, x_j, edge_attr):
        # x_i = receiver features, x_j = sender features
        return self.edge_mlp(torch.cat([x_i, x_j, edge_attr], dim=-1))

    def update(self, aggr_out, x):
        return self.node_mlp(torch.cat([x, aggr_out], dim=-1))

```

PyG concept	Meaning
<code>edge_index</code>	(2, E) long tensor, [source; target], directed
<code>x_i / x_j</code> in message	<code>_i</code> = target/receiver, <code>_j</code> = source/sender
<code>aggr</code>	"add", "mean", "max", or a custom Aggregation
Batch	Disjoint union of graphs; <code>batch.batch</code> gives the graph index per node
<code>global_mean_pool(x, batch)</code>	Graph-level readout
<code>scatter(src, index, dim, reduce)</code>	The primitive underneath all aggregation

10.5 Related architectures you should be able to name

Model	Idea	Good for
GraphNet / MeshGraphNet	Encode-process-decode with edge+node updates	Unstructured transient simulation
MultiScale MGN / Hierarchical	Coarse graph levels, multigrid-style	Long-range pressure coupling
Fourier Neural Operator (FNO)	Learn in Fourier space; resolution-invariant operator	Regular grids, periodic domains
DeepONet	Branch net (input function) x trunk net (query point)	Operator learning, sparse sensors
GraphSAGE / GAT / GCN	Generic node-classification GNNs	Baselines; GAT for attention weighting
Transolver / attention-based	Attention over physical tokens	Large-scale automotive external aero
PointNet / point-cloud nets	Permutation-invariant on raw points	Surface pressure prediction, no connectivity

Question: “MeshGraphNet or FNO for external vehicle aerodynamics?” FNO wants a regular grid; a car surface is unstructured with huge scale separation. MGN handles the mesh natively, but pressure is globally coupled and pure local message passing needs many layers or a hierarchy. A good answer names the trade-off and proposes a hierarchical/multiscale GNN, or a graph-transformer, rather than picking one dogmatically.

10.6 Practice set — Part 10

Q10.1 Implement `index_add_`-based scatter aggregation and verify it against a slow loop. Then implement mean and max aggregation from the same primitive.

Q10.2 Build the full encode-process-decode network and train it on a toy problem: predict node-wise heat diffusion increments on a random triangular mesh. Verify that the trained model generalises to a mesh with a different number of nodes.

Q10.3 Explain and implement why edge features must use relative rather than absolute positions. Demonstrate the failure by training with absolute positions and testing on a translated mesh.

Q10.4 Implement the noise-injection training strategy and compare 100-step rollout error with and without it.

Q10.5 Given a batched PyG `Batch` object, compute a per-graph mean of a node field without using `global_mean_pool`. Use `scatter` with `batch.batch`.

Q10.6 Your rollout is stable but the total mass drifts by 3% over 200 steps. Give three ways to fix it. *Answers: predict fluxes on edges with antisymmetry so they cancel pairwise; add a projection/correction step after each rollout step; add a conservation penalty to the loss.*

Part 11 — ML Engineering for Scientific Data

11.1 Splitting data correctly — the most common take-home failure

Random splitting of snapshots from the same simulation leaks information: adjacent time steps are nearly identical, so validation error is meaninglessly low.

Task	Correct split
Time series from one run	Split by time block (train on early, validate on late)
Multiple simulation cases	Split by case/geometry , never by snapshot
Parametric study	Hold out whole parameter values, and test at least one extrapolation point
Same geometry, different Re	Hold out Re values, and state clearly which are interpolation and which are extrapolation

```
import numpy as np

def group_split(case_ids: np.ndarray, frac=(0.7, 0.15, 0.15), seed=0):
    """Split by group so no case appears in two sets."""
    rng = np.random.default_rng(seed)
    groups = rng.permutation(np.unique(case_ids))
    n1 = int(frac[0] * len(groups)); n2 = n1 + int(frac[1] * len(groups))
    tr, va, te = groups[:n1], groups[n1:n2], groups[n2:]
    idx = lambda g: np.flatnonzero(np.isin(case_ids, g))
    return idx(tr), idx(va), idx(te)
```

11.2 Normalisation

```
class StandardScaler:
    """Fit on TRAIN ONLY. Fitting on all data is textbook leakage."""
    def fit(self, x, axis=0):
        self.mean_ = x.mean(axis, keepdims=True)
        self.std_ = np.maximum(x.std(axis, keepdims=True), 1e-8)
        return self
    def transform(self, x): return (x - self.mean_) / self.std_
    def inverse(self, x): return x * self.std_ + self.mean_
```

Scheme	Use when
Z-score (standard)	Default for network inputs and targets
Min-max to [-1, 1]	Bounded quantities, tanh activations
Log transform	Positive quantities spanning decades (k, epsilon, nu_t)
Non-dimensionalisation	Always preferred — divide by U_ref, L_ref, rho U^2. Gives physical generality that statistical scaling does not
Per-component vs global	Vector fields: use a single scale for all components to preserve isotropy

Say this in the interview: normalising by $0.5 \rho U^2$ to get C_p is better than z-scoring pressure, because the model then transfers to a different inlet velocity without retraining.

11.3 Metrics for field prediction


```

import numpy as np

def rel_l2(pred, true, eps=1e-12):
    return np.linalg.norm(pred - true) / (np.linalg.norm(true) + eps)

def per_node_rmse(pred, true):
    return np.sqrt(((pred - true) ** 2).mean(axis=0))

def integrated_quantity_error(p_pred, p_true, area, normal, direction):
    """The metric an engineering manager actually cares about: does the drag match?"""
    f_pred = np.einsum("nj,nj->", (p_pred[:, None] * normal) * area[:, None],
                       np.broadcast_to(direction, normal.shape))
    f_true = np.einsum("nj,nj->", (p_true[:, None] * normal) * area[:, None],
                       np.broadcast_to(direction, normal.shape))
    return abs(f_pred - f_true) / abs(f_true)

```

Metric	What it tells you
MSE / RMSE	Overall point-wise fit; dominated by large-magnitude regions
Relative L2	Scale-free; the standard in the PINN/neural-operator literature
MAE	Robust to outliers; better when a few cells have extreme values
R ²	Fraction of variance explained; misleading for fields with low variance
Integrated quantities (Cd, Cl, Nu, Δp)	The engineering acceptance criterion — always report these
Spectral error	Whether small scales are captured or smeared
Rollout error vs horizon	Stability of an autoregressive model
Conservation residual	Does the prediction respect mass/energy balance

11.4 Reproducibility

```

import os, random, numpy as np, torch

def set_seed(seed: int = 0, deterministic: bool = True):
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
    if deterministic:
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False
        torch.use_deterministic_algorithms(True, warn_only=True)
    os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":4096:8"

```

Also: pin dependency versions, log the git commit hash, save the config alongside the checkpoint.

```

torch.save({"model": model.state_dict(),
           "optim": opt.state_dict(),
           "epoch": epoch,
           "config": cfg,
           "normalizer": scaler.__dict__,
           "git_sha": subprocess.check_output(["git", "rev-parse", "HEAD"]).decode().strip(),
           "ckpt.pt"})

```

11.5 Overfitting, underfitting and what to do

Observation	Diagnosis	Action
Train loss low, val loss high and rising	Overfitting	More data/augmentation, weight decay, dropout, early stopping, smaller model
Both losses high and flat	Underfitting / optimisation failure	Bigger model, longer training, higher LR, check normalisation, check the loss is differentiable
Loss = NaN	Exploding gradients, <code>log(0)</code> , <code>sqrt(negative)</code> , LR too high	Gradient clipping, lower LR, <code>eps</code> guards, check for NaN in data
Val loss much <i>lower</i> than train	Data leakage, or dropout still active in eval	Check the split; call <code>model.eval()</code>
Great on interpolation, terrible outside the range	Expected — networks do not extrapolate	State the validity envelope; embed physics; add data

11.6 Practice set — Part 11

Q11.1 Given a dataset of 50 simulations at different Reynolds numbers, write the split logic and justify it in three sentences.

Q11.2 Implement a `fit/transform/inverse_transform` scaler that handles a vector field isotropically and a set of scalar fields component-wise.

Q11.3 Write an evaluation function that reports relative L2, per-field RMSE, and the drag coefficient error, returning a tidy DataFrame.

Q11.4 Your validation loss is 100x lower than your test loss. List the five things you check, in order.

Part 12 — Performance, Testing and Engineering Practice

12.1 Profiling before optimising

```
import cProfile, pstats
cProfile.run("main()", "prof.out")
pstats.Stats("prof.out").sort_stats("cumtime").print_stats(20)
```

```
python -m cProfile -o prof.out script.py
pip install line_profiler # then decorate with @profile and run kernprof -l -v script.py
pip install memory_profiler # @profile for line-by-line memory
```

```
# PyTorch profiler - shows CPU/GPU split and which kernels dominate
from torch.profiler import profile, ProfilerActivity
with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
             record_shapes=True) as prof:
    train_one_epoch()
print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=15))
```

12.2 Speed-up options, in the order you should reach for them

1. **Vectorise** — remove Python loops. Usually 10–1000x. Free.
2. **Better algorithm** — sparse instead of dense, KD-tree instead of $O(n^2)$, `argpartition` instead of full `argsort`.
3. **Avoid copies** — in-place ops, views, preallocation, correct dtype.
4. **Numba** — `@njit`, `@njit(parallel=True)` with `prange`, for loops that genuinely cannot be vectorised (stencil kernels with complex branching).
5. **Cython / C++ extension** — when Numba is not enough.
6. **Multiprocessing** — CPU-bound parallelism across cases (the GIL blocks threads).
7. **GPU** — CuPy for array code, PyTorch for anything differentiable.

```
from numba import njit, prange

@njit(parallel=True, fastmath=True, cache=True)
def jacobi_sweep(p, b, dx2, dy2):
    ny, nx = p.shape
    pn = p.copy()
    for j in prange(1, ny - 1):
        for i in prange(1, nx - 1):
            p[j, i] = ((pn[j, i+1] + pn[j, i-1]) * dy2
                       + (pn[j+1, i] + pn[j-1, i]) * dx2
                       - b[j, i] * dx2 * dy2) / (2.0 * (dx2 + dy2))

    return p
```

```
from concurrent.futures import ProcessPoolExecutor

with ProcessPoolExecutor(max_workers=8) as ex:
    results = list(ex.map(run_single_case, case_list))
```

GIL question: the Global Interpreter Lock allows only one thread to execute Python bytecode at a time, so `threading` does not speed up CPU-bound Python. It *does* help for I/O-bound work, and NumPy/PyTorch release the GIL inside their C kernels, so BLAS-heavy code is already multi-threaded. For CPU-bound pure-Python work, use `multiprocessing`.

12.3 Testing numerical code

```

import numpy as np
import pytest

def test_laplacian_on_linear_field_is_zero():
    """A linear field has zero second derivative - exact to round-off."""
    x = np.linspace(0, 1, 65)
    X, Y = np.meshgrid(x, x, indexing="ij")
    phi = 3.0 * X + 2.0 * Y
    lap = laplacian(phi, h=x[1] - x[0])
    assert np.max(np.abs(lap[1:-1, 1:-1])) < 1e-10

@pytest.mark.parametrize("n", [17, 33, 65, 129])
def test_second_order_convergence(n):
    err = poisson_error(n)
    assert err < 5.0 / n**2 # C/h^2 bound

def test_conservation():
    """Total mass must be preserved to machine precision."""
    m0 = (rho * vol).sum()
    m1 = (advance(rho) * vol).sum()
    assert np.isclose(m0, m1, rtol=1e-12)

def test_gradient_matches_finite_difference():
    """Standard check for a custom autograd op."""
    x = torch.randn(10, 3, dtype=torch.double, requires_grad=True)
    assert torch.autograd.gradcheck(my_op, (x,), eps=1e-6, atol=1e-4)

```

Categories of test that reviewers look for in a take-home:

- **Analytic solution** — compare against an exact case.
- **Symmetry / invariance** — translate, rotate, permute the input; the output must transform correspondingly (crucial for GNNs).
- **Conservation** — mass, momentum, energy.
- **Convergence order** — MMS or grid refinement.
- **Shape and dtype contracts** — cheap, catches most bugs.
- **Regression** — pin a known-good output hash.

12.4 Project layout and tooling

```

project/
├── pyproject.toml           # deps, build config, tool settings
├── README.md               # what, how to run, results, assumptions, limitations
├── configs/
│   └── baseline.yaml
├── src/pkg/
│   ├── __init__.py
│   ├── data.py             # loading, splitting, normalisation
│   ├── models.py           # nn.Module definitions only
│   ├── physics.py          # residuals, derivatives
│   ├── train.py            # loop, checkpointing, logging
│   └── evaluate.py         # metrics, plots
├── tests/
│   └── test_physics.py
├── scripts/
│   └── run_experiment.py

```

Mention `ruff` (lint + format), `mypy` (types), `pytest`, `pre-commit`, `hydra/OmegaConf` (config), `mlflow/wandb/tensorboard` (experiment tracking). Naming two or three of these signals professional practice.

12.5 Practice set — Part 12

Q12.1 Profile a naive $O(n^2)$ nearest-neighbour search and replace it with a KD-tree. Report the speed-up at $n = 10^3, 10^4, 10^5$.

Q12.2 Write a `pytest` suite for your Part 6 Poisson solver covering: linear field exactness, symmetry, second-order convergence, and a shape contract.

Q12.3 Convert a nested-loop upwind advection kernel to (a) NumPy vectorised, (b) Numba jitted. Benchmark all three.

Q12.4 Write a test proving your GNN is permutation-equivariant: shuffle node ordering, remap the edge

index, and assert the outputs match after unshuffling.

Part 13 — Conceptual Question Bank with Model Answers

Rapid-fire questions from real technical rounds for scientific-ML roles. Read the question, answer out loud, then check.

13.1 Python and software

Q. What is the GIL and when does it hurt you? One thread executes Python bytecode at a time. It hurts CPU-bound pure-Python parallelism. It does not hurt I/O-bound work, nor NumPy/PyTorch kernels that release it. Use `multiprocessing` for CPU-bound work, threads for I/O.

Q. `is` vs `==`? `is` compares identity (same object), `==` compares value via `__eq__`. Use `is` only for singletons (`None`, `True`, `False`).

Q. Shallow vs deep copy? `copy.copy` duplicates the container but shares the nested objects; `copy.deepcopy` recursively duplicates everything. For NumPy, `arr.copy()` is a full data copy; a slice is a view.

Q. How does Python manage memory? Reference counting plus a cyclic garbage collector for reference cycles. `del` decrements the count; it does not necessarily free memory immediately.

Q. What is a `__iter__`/`__next__` protocol and how does `yield` relate? An iterable returns an iterator from `__iter__`; the iterator implements `__next__` and raises `StopIteration`. A generator function is a compiler-generated iterator; `yield` suspends and resumes execution while preserving local state.

13.2 NumPy and numerics

Q. When does NumPy copy versus view? Basic slicing gives a view; fancy/boolean indexing gives a copy; `reshape` gives a view when strides permit. Check with `arr.base`.

Q. Why avoid `np.linalg.inv`? Forming the inverse is more expensive and numerically worse than solving the system directly. Use `solve`, or a factorisation you reuse (`scipy.linalg.lu_factor` / `cho_factor`).

Q. What is the condition number and why do you care? $\text{cond}(A) = \|A\| \|A^{-1}\|$; it bounds how much relative input error is amplified in the solution. A condition number of 10^k costs roughly k digits of accuracy. Fine grids and high aspect-ratio cells make CFD matrices badly conditioned — hence preconditioning.

Q. float32 vs float64 in scientific ML? float32 halves memory and roughly doubles GPU throughput, but has ~ 7 significant digits. Fine for network weights and inference; risky for residual computation, accumulation over many steps, or ill-conditioned linear algebra. Common compromise: float32 network, float64 for evaluation metrics and any accumulation.

13.3 CFD and numerics

Q. Explain the CFL condition physically. The numerical domain of dependence must contain the physical one. Information travels at most one cell per time step in an explicit scheme; if the physical wave moves further, the scheme cannot see the information it needs and blows up.

Q. Why is first-order upwind stable but bad? Its truncation error is a diffusion term with $\text{nu_num} = |u| \Delta x / 2$, which damps oscillations — hence stability — but also smears gradients. Godunov's theorem: no linear scheme above first order is monotone, so you need nonlinear limiters (TVD/MUSCL/WENO).

Q. RANS vs LES vs DNS, in one sentence each. RANS time-averages and models all turbulent scales — cheapest, model-dependent. LES resolves the energy-containing scales and models the subgrid — expensive, grid-dependent through the filter. DNS resolves everything down to Kolmogorov — cost scales roughly as Re^3 , research only.

Q. What is y^+ and why does it matter for ML on CFD data? Non-dimensional wall distance $y^+ = u_\tau y / \nu$. It determines whether the first cell resolves the viscous sublayer or relies on a wall function. If your training data mixes wall-resolved and wall-modelled cases, near-wall quantities are not comparable — a real dataset-hygiene issue.

Q. Why is pressure-velocity coupling hard? There is no explicit pressure equation in incompressible flow; pressure acts as a Lagrange multiplier enforcing $\nabla \cdot \mathbf{u} = 0$. Hence SIMPLE/PISO/projection methods, and hence the elliptic (globally coupled, infinite-speed) nature of the pressure field — which is exactly why local message-passing GNNs struggle with pressure.

13.4 Machine learning

Q. Bias-variance trade-off? Bias is systematic error from an over-restrictive model; variance is sensitivity to the particular training sample. More capacity lowers bias and raises variance. Regularisation, more data and ensembling reduce variance.

Q. Batch norm vs layer norm, and which for GNNs? BatchNorm normalises across the batch dimension per feature — it depends on batch composition and behaves differently at train and eval. LayerNorm normalises across features per sample — batch-independent. GNNs and transformers use LayerNorm because graph batch sizes and node counts vary; MeshGraphNet uses LayerNorm at the end of each MLP.

Q. Why does Adam usually beat SGD here? Per-parameter adaptive step sizes handle the wildly different gradient scales of multi-term physics losses. Downside: it can converge to sharper minima and is less well-behaved for very long training; AdamW fixes the weight-decay coupling.

Q. What is the universal approximation theorem and what does it *not* say? A sufficiently wide network can approximate any continuous function on a compact set to arbitrary accuracy. It says nothing about how many parameters, whether optimisation will find them, or anything about generalisation or extrapolation. Useful as a sanity check, useless as a guarantee.

Q. What is spectral bias? Networks trained with gradient descent learn low frequencies first and high frequencies very slowly. It is why PINNs struggle with shocks and boundary layers, and why Fourier features, sinusoidal activations (SIREN) and domain decomposition help.

13.5 PINN and operator learning

Q. Where do the derivatives come from in a PINN? Automatic differentiation of the network with respect to its inputs — exact to machine precision, mesh-free. The cost is that each derivative order requires another backward pass through the graph, so second derivatives in 3-D are expensive.

Q. Difference between a PINN and a neural operator? A PINN learns *one solution field* for one set of conditions; retraining is needed for new conditions. A neural operator (FNO, DeepONet) learns a *mapping between function spaces*, so inference on a new input function is a single forward pass. Operators need training data; PINNs can work with none.

Q. Can PINNs handle turbulence? Not well for forward high-Re turbulence: the solution has a broad spectrum, spectral bias fights you, and the residual is stiff. They are used successfully for RANS-level closure, data assimilation of experimental fields, and inverse problems.

Q. How do you impose a Neumann condition in a PINN? Compute the normal derivative by autograd at boundary points and penalise $(\partial u / \partial n - g)^2$; or build it into the ansatz. For a pure-Neumann problem, pin the level with an integral constraint or a single reference point, otherwise the solution is non-unique.

13.6 Graph networks

Q. Why sum aggregation for physics? Sum is the discrete analogue of a flux balance over the faces of a control volume and preserves extensive quantities; mean discards magnitude and degree information. Sum is also the most expressive of the simple aggregators (see the GIN/WL-test result).

Q. What symmetries should a mesh-based surrogate respect? Permutation equivariance over nodes (automatic in a GNN), translation invariance (use relative positions), and ideally rotation equivariance (achieved by using invariant edge features such as distances, or by an equivariant architecture such as E(3)-GNN / SE(3)-transformer).

Q. Why does an autoregressive surrogate diverge over long rollouts? Exposure bias: the model is trained on ground-truth inputs but consumes its own outputs at inference. Errors compound geometrically. Mitigations: training-noise injection with corrected targets, multi-step (pushforward) training, scheduled sampling, and re-imposing boundary conditions each step.

Q. How would you scale a GNN to a 50-million-cell automotive mesh? Graph partitioning with halo exchange across GPUs; hierarchical/multiscale graphs so global coupling does not need 50 message-passing layers; subsampling or surface-only graphs for aerodynamic coefficients; mixed precision and activation checkpointing for memory. Honest answer: you almost always coarsen or work on the surface mesh.

13.7 Judgement and the questions with no code

Q. Your surrogate is 99% accurate on the test set. Would you ship it? Not on that number alone. I would check: is the test split by geometry or by snapshot; what is the error on the *integrated* quantity the engineers use; how does it behave outside the training envelope; are there failure cases with physically

impossible output; and is there a cheap uncertainty estimate or fallback to the solver when the input looks out of distribution.

Q. How do you quantify uncertainty? Deep ensembles (simplest and strongest baseline), MC dropout, quantile regression, or a conservative out-of-distribution detector on the input features. Also report the residual of the governing equations evaluated on the prediction — a physics-based confidence measure that is often more useful than a statistical one.

Q. An engineer says they do not trust the model. What do you do? Show validation against cases they already believe, on the metric they care about (C_d , Δp , peak temperature), with the error envelope stated explicitly. Give them the residual/uncertainty diagnostic so they can see when to distrust it. Position the surrogate as a screening tool that narrows the design space before high-fidelity confirmation, not as a replacement.

Part 14 — Timed Mock Tests

Do these under exam conditions: no autocomplete assistance, a timer running, and write the tests before you write the implementation where noted.

Mock Test A — Automated screen (60 minutes)

A1 (10 min). Given a list of dictionaries describing simulation runs `{"case": str, "Re": float, "cd": float, "converged": bool}`, return the case name with the lowest `cd` among converged runs with `Re` between `1e5` and `1e6`. Return `None` if there are none.

A2 (10 min). Given a string of solver log lines, extract every residual value matching `Ux: 1.234e-05` and return a dict of field name to list of residuals. Use `re`.

A3 (15 min). Given an integer array of cell zone ids, return the size of the largest contiguous run of the same zone, and the zone id. One pass, $O(1)$ extra memory.

A4 (25 min). Given `points (N,3)` and `radius`, return a list of index pairs closer than `radius`. Provide a naive version, then an $O(N \log N)$ version, and a test that they agree.

Mock Test B — Live pair-coding (45 minutes)

You will be asked to build this incrementally while narrating.

"Implement a small physics-informed network that solves the 1-D steady convection-diffusion equation $u \phi' = \Gamma \phi''$ on $[0, 1]$ with $\phi(0)=0$, $\phi(1)=1$. Then compare against the analytic solution $\phi = (\exp(\text{Pe} x) - 1) / (\exp(\text{Pe}) - 1)$ with $\text{Pe} = uL/\Gamma$. Start with `Pe = 1`, then push to `Pe = 50` and tell me what happens."

Checkpoints the interviewer is scoring:

1. Correct autograd second derivative with `create_graph=True` (5 min).
2. A working training loop, loss decreasing (15 min).
3. Correct error metric versus the analytic solution (5 min).
4. **The `Pe = 50` discussion** — the boundary layer at $x=1$ has thickness $\sim 1/\text{Pe}$; a plain MLP under-resolves it due to spectral bias. Fixes: hard-constraint ansatz, Fourier features, collocation points clustered near $x=1$, or non-dimensionalising the residual. Reaching this discussion matters more than a perfect implementation.

```
# Reference solution sketch
def residual(model, x, u=1.0, gamma=1.0/50):
    phi = model(x)
    phi_x = grad(phi, x)
    phi_xx = grad(phi_x, x)
    return u * phi_x - gamma * phi_xx

def hard_ansatz(model, x):
    """phi = x + x(1-x) * NN -> phi(0)=0, phi(1)=1 exactly."""
    return x + x * (1 - x) * model(x)
```

Mock Test C — Take-home (4 hours)

Task. You are given `dataset.h5` containing 60 two-dimensional steady flow cases around randomly generated obstacles. Each case has: node coordinates, triangular connectivity, node type, inlet velocity, and the converged velocity and pressure fields.

Build a model that predicts the velocity and pressure field given the geometry and inlet velocity. Evaluate it. Submit code plus a short report.

A strong submission contains:

Element	Why it scores
README.md : how to run, results table, assumptions, limitations	Communication is 15–20% of the rubric
Split by case , with an explicit statement about interpolation vs extrapolation	Shows you understand leakage
Non-dimensionalisation (u/U_{in} , $p/(0.5 \rho U_{in}^2)$)	Physical insight, and it makes the model transferable
A trivial baseline (predict the mean field, or a nearest-neighbour case)	Proves the learned model actually adds value
GNN with relative-position edge features	The core competence being tested
Metrics: relative L2 and an integrated quantity (drag on the obstacle)	Engineering relevance
One figure: predicted vs true field plus the error map for the worst test case	Honesty about failure modes
A tests/ folder with at least a permutation-equivariance test	Rare, and it stands out
Explicit statement of what you would do with more time	Shows judgement and scoping

Deliberate traps in this kind of task:

- Snapshots from the same case in both train and test (leakage).
- Feeding absolute coordinates as node features (breaks generalisation).
- Reporting only MSE on normalised data (looks great, means nothing).
- No baseline, so a 0.05 relative L2 has no context.
- Training for 8 hours and delivering no analysis. Budget half your time for evaluation and writing.

Mock Test D — Whiteboard (30 minutes)

1. Derive the discrete Laplacian stencil in 1-D from Taylor series and state its order of accuracy.
2. Write the PINN loss for the incompressible Navier-Stokes equations, naming every term.
3. Sketch the MeshGraphNet architecture and explain what each MLP consumes and produces.
4. You train a surrogate and the rollout diverges after 40 steps. Walk through your debugging.
5. Your PINN converges to $u \equiv 0$. What are the three most likely causes?

Part 15 — Cheat Sheets and Study Plan

15.1 Numbers to have memorised

Quantity	Value
float64 epsilon	2.22e-16 (≈ 16 digits)
float32 epsilon	1.19e-07 (≈ 7 digits)
CFL limit, explicit convection	≤ 1
Diffusion number limit, explicit	0.5 (1-D), 0.25 (2-D), 1/6 (3-D)
Cell Peclet limit, central differencing	2
DNS cost scaling	$\sim \text{Re}^3$
Kolmogorov length scale	$\eta = (\nu^3/\epsilon)^{1/4}$
Typical MeshGraphNet depth	15 message-passing blocks, hidden 128
Typical PINN	4–8 hidden layers, 20–128 wide, tanh, Adam 1e-3 then L-BFGS

15.2 Complexity table

Operation	Complexity
Dict/set lookup	$O(1)$ average
Sort	$O(n \log n)$
Heap push/pop	$O(\log n)$
Dense LU solve	$O(n^3)$
Sparse CG iteration	$O(\text{nnz})$ per iteration
KD-tree build / query	$O(n \log n)$ / $O(\log n)$
Naive pairwise distance	$O(n^2)$
Message passing layer	$O(E)$
Dense SVD	$O(\min(m,n)^2 \max(m,n))$

15.3 One-page PyTorch recall list

```
torch.cat / stack / chunk / split / unsqueeze / squeeze / permute / reshape
torch.einsum("nij,nj->ni", A, b)
torch.autograd.grad(y, x, torch.ones_like(y), create_graph=True)[0]
tensor.index_add_(0, idx, src) # scatter-add
tensor.scatter_add_(0, idx_expanded, src)
torch.nn.functional.one_hot(labels, n)
torch.nn.utils.clip_grad_norm_(params, 1.0)
optimizer.zero_grad(set_to_none=True)
with torch.no_grad(): ... # inference
model.train() / model.eval() # dropout & batchnorm behaviour
torch.save(state_dict) / model.load_state_dict(torch.load(path))
torch.cuda.amp.autocast() + GradScaler # mixed precision
```

15.4 Four-week preparation plan

Week	Focus	Daily commitment	Deliverable
1	Parts 1–3. Core Python, NumPy, SciPy. Do every practice question by hand.	1.5 h theory + 1 h coding	A personal <code>numpy_recipes.py</code> with 30 vectorised idioms
2	Part 5 DSA (30 problems: 15 easy, 15 medium, weighted to hash maps, two pointers, BFS/DFS, binary search) + Part 6 numerics	1 h DSA + 1 h numerics	1-D advection, Burgers and 2-D cavity solvers, all with MMS verification
3	Parts 8–9. PyTorch and PINNs. Type every code block from memory, do not copy.	2 h	A working Burgers PINN plus the inverse-problem variant, in your own repo
4	Part 10 GNNs + Parts 11–13 + mock tests	2 h	A MeshGraphNet on a toy mesh dataset; Mock Test C completed end to end

Final 48 hours: re-read Parts 13 and 15 only. Re-run your own Burgers PINN and toy GNN so the code is fresh in your fingers. Prepare a three-minute spoken description of one project where you connected CFD and ML — they will ask, and a crisp answer sets the tone for everything after it.

15.5 Ten things to say that signal seniority

1. “Let me non-dimensionalise this first.”
2. “I’d verify with the method of manufactured solutions before trusting any of it.”
3. “I’ll split by case, not by snapshot, otherwise the validation number is meaningless.”
4. “Sum aggregation, because it mirrors a flux balance.”
5. “Relative positions only on the edges — absolute coordinates would kill generalisation.”
6. “I’d start with a trivial baseline so we know what the model is actually buying us.”
7. “The engineering metric here is drag, not MSE, so let me report both.”
8. “This needs `create_graph=True`, otherwise the physics loss contributes no gradient.”
9. “That’s an extrapolation, and networks don’t extrapolate — I’d flag the validity envelope.”
10. “I don’t know, but here’s how I’d find out.”

15.6 Self-assessment checklist

Tick only what you can do **from memory, without reference**:

- ☐ Write a decorator with arguments
- ☐ Explain view vs copy and predict which a given NumPy op returns
- ☐ Vectorise a scatter-add onto mesh nodes and explain why `a[idx] += x` is wrong
- ☐ Assemble a sparse Laplacian and solve with preconditioned CG
- ☐ Write the 1-D Burgers solver with correct CFL and diffusion limits
- ☐ State the diffusion-number limit in 3-D
- ☐ Build a bidirectional, deduplicated edge list from a triangle mesh
- ☐ Compute a second derivative with `torch.autograd.grad`
- ☐ Write the Burgers PINN loss including hard-constraint alternatives
- ☐ Explain three reasons a PINN converges to the trivial solution
- ☐ Write a message-passing layer from scratch with `index_add_`
- ☐ Explain the MeshGraphNet noise-injection trick and why the target is adjusted
- ☐ Design a leakage-free split for a parametric CFD dataset
- ☐ Name the metric an engineering stakeholder actually cares about
- ☐ Write a permutation-equivariance test for a GNN

Anything unticked two weeks before the interview is your revision list.